
THE EVOLUTION OF MACHINE LEARNING AND AI · WEEK 1 OF 12

The Shape of the Field

Cycles, breakthroughs, and what the current wave got right

Live lecture · Interactive timeline · 45-page booklet · Week 6 quiz (50%, LO1)

75 YEARS IN 2 HOURS

Today



One pattern that repeats.

One pattern that repeats. **One question:** why did brute force suddenly work in 2012?

One pattern that repeats. **One question:** why did brute force suddenly work in 2012? **One thesis** to carry for twelve weeks:
scaling worked · **scaling is hitting walls** · **the field is turning.**

One pattern that repeats. **One question:** why did brute force suddenly work in 2012? **One thesis** to carry for twelve weeks:
scaling worked · **scaling is hitting walls** · **the field is turning.**

History is not background here. It is a **user manual for the present:** this month's headlines had near-identical twins in 1958, 1970, and 1985.

Your materials this week

Slides

What we work through together in the two hours. Every build step is its own page, so the PDF replays exactly what the room saw.

Booklet

The full account: chapters, the master timeline, a glossary, and 43 practice questions in the Week 6 registers.

Code

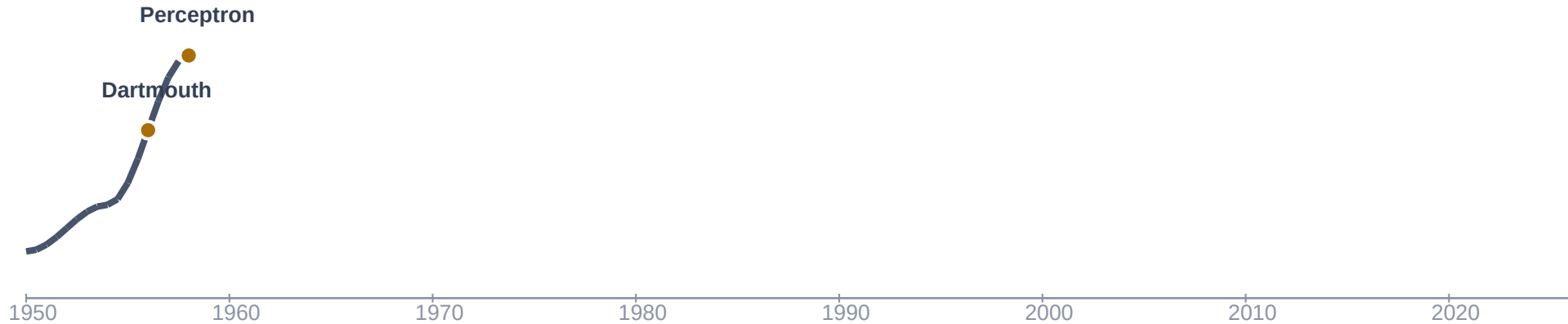
`week1-lab/`, a runnable folder: an environment check, the timeline as a dataset, and the module thesis as an experiment.

REPRODUCIBILITY FROM DAY ONE

Every number and table on the code slides later in this deck came out of that folder, at a fixed seed, with no hand-tuning. Clone it and you get the same results. **If you cannot reproduce a claim, it is not yet a result.**

The one picture to remember

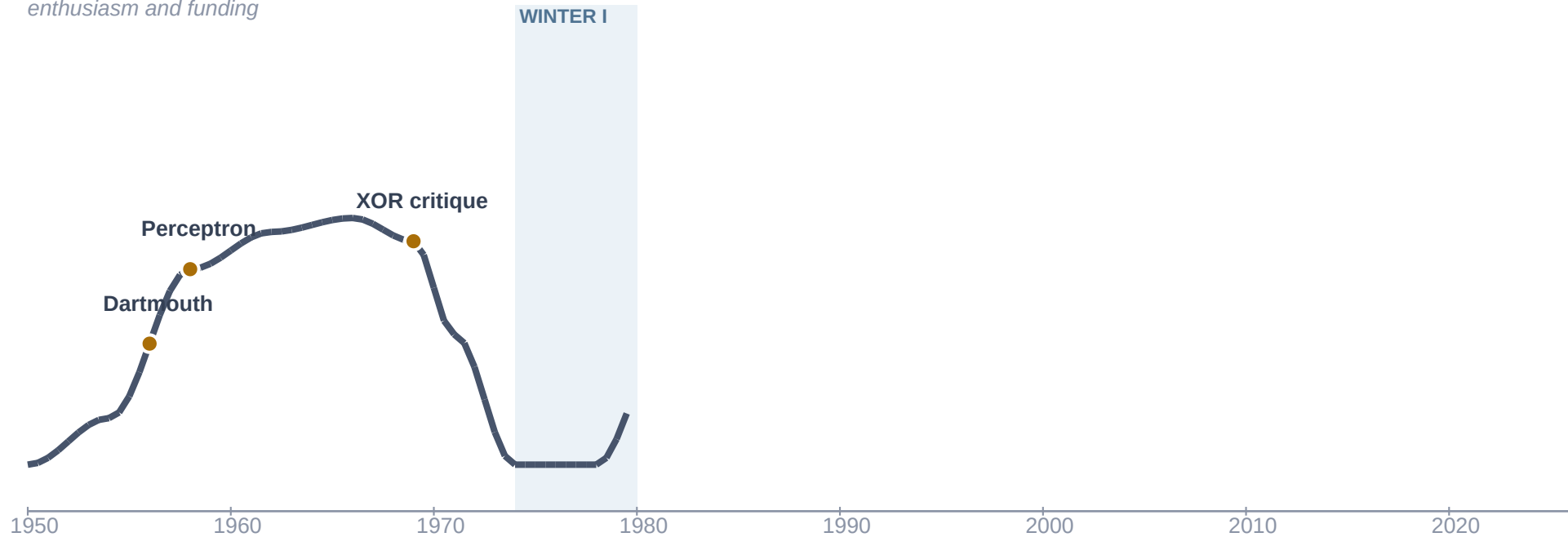
enthusiasm and funding



A real breakthrough starts every climb...

The one picture to remember

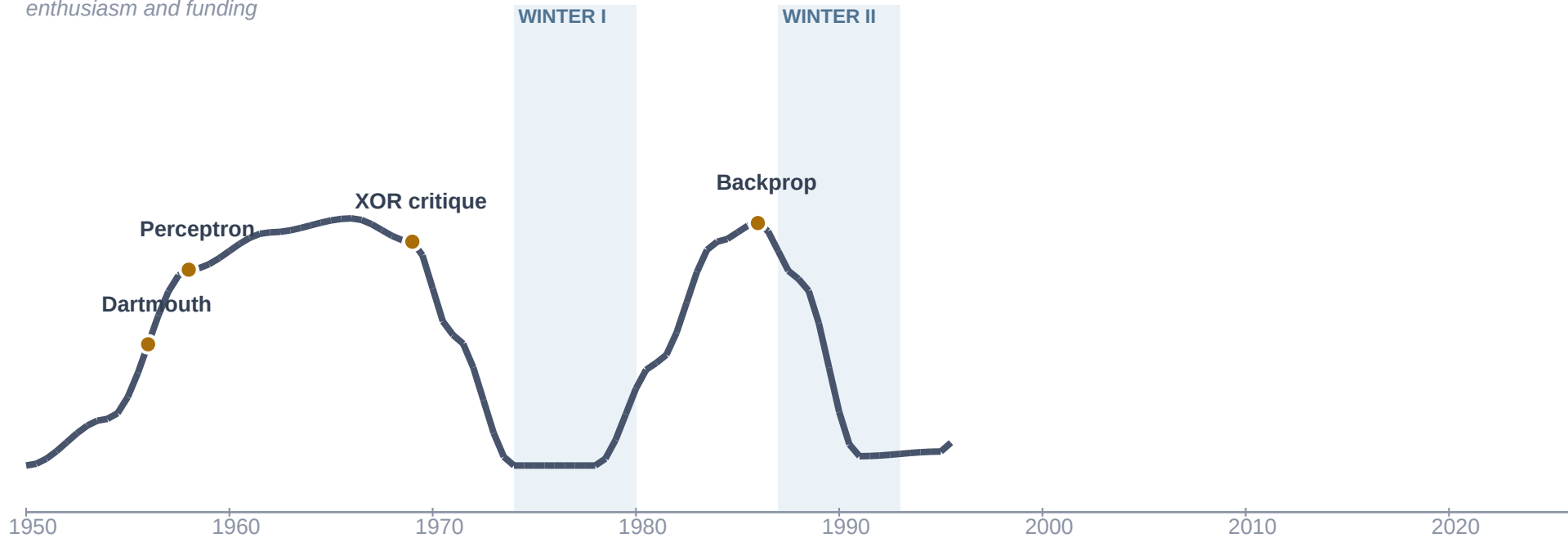
enthusiasm and funding



...promises outrun results: **winter.**

The one picture to remember

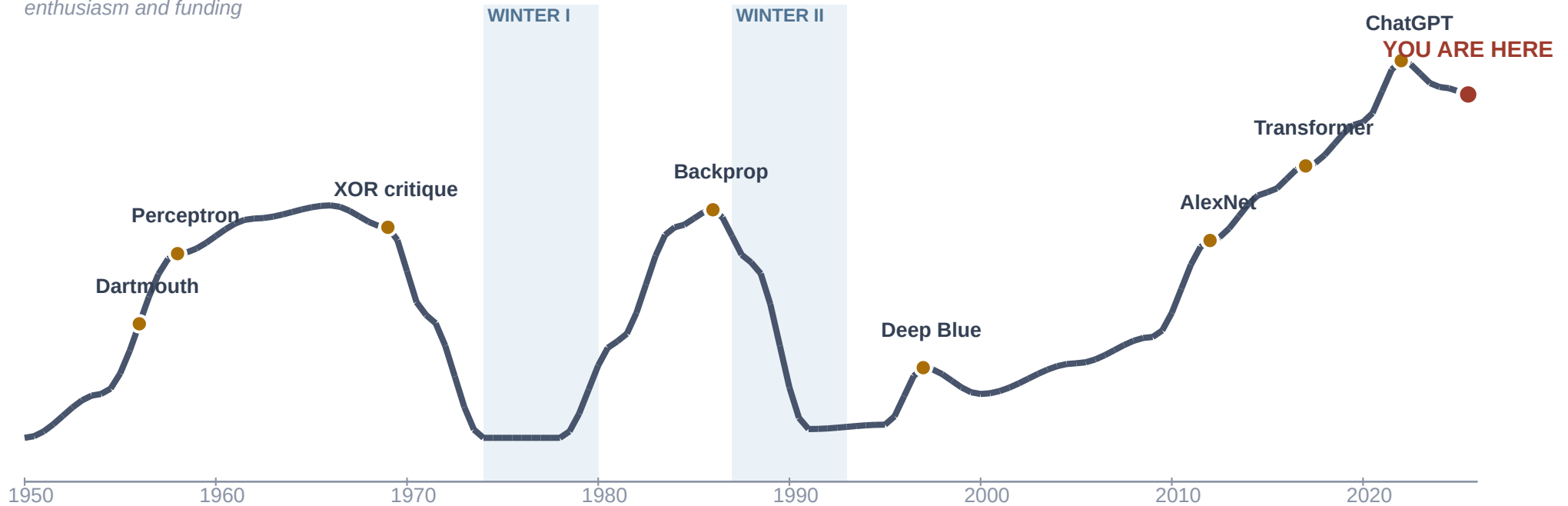
enthusiasm and funding



A narrower promise rebuilds. Same gap. **Winter II**. The field even changes its name.

The one picture to remember

enthusiasm and funding



The third rise is the biggest yet. **You are studying the field near its top.**

The seaside town

EVERYDAY ANALOGY

A genuinely good beach. Speculators build **more hotels than the beach can fill**. One bad season: bankruptcies, and the town is called a failure. **The beach was real all along.**

The seaside town

EVERYDAY ANALOGY

A genuinely good beach. Speculators build **more hotels than the beach can fill**. One bad season: bankruptcies, and the town is called a failure. **The beach was real all along.**

Standing question for the semester:

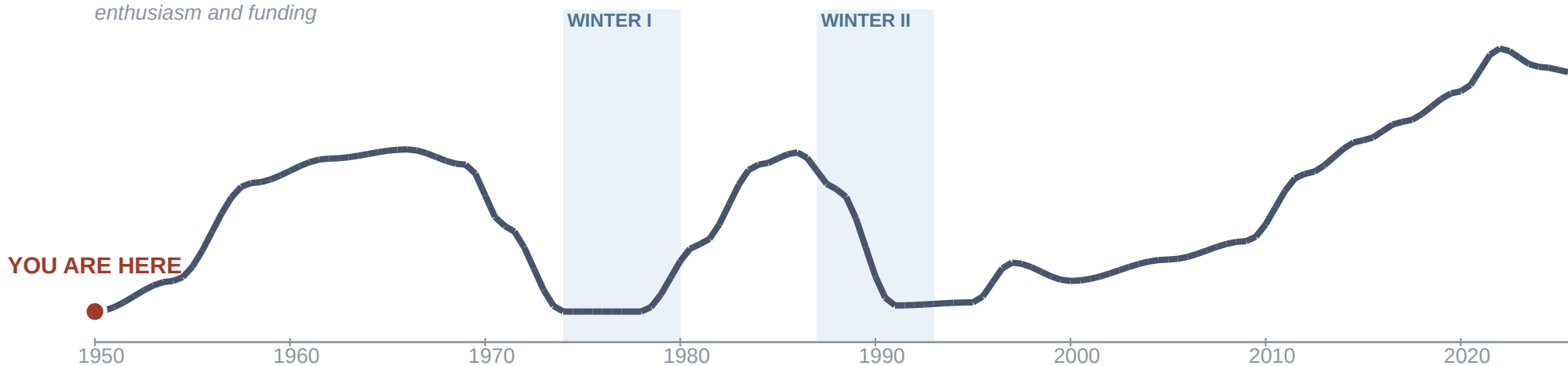
*is this claim about the **beach**, or about the **hotels**?*

PART 1

Before the beginning

Myths, mathematics, and a dangerous inference · to 1949

enthusiasm and funding



1843: the first objection



Ada Lovelace argued that the machine could only do what humans knew how to instruct it to do.

*“ The Engine has no pretensions whatever to **originate** anything. It can do whatever we **know how to order it** to perform. “*

Ada Lovelace, notes on the Analytical Engine

1843: the first objection



Ada Lovelace argued that the machine could only do what humans knew how to instruct it to do.

*“ The Engine has no pretensions whatever to **originate** anything. It can do whatever we **know how to order it** to perform. “*

Ada Lovelace, notes on the Analytical Engine

You will hear this at dinner this week: *“AI just remixes its training data.”* Hold it until a machine plays **move 37**.

By 1949, the inference is complete

Boole, 1854
reasoning is **algebra**

Shannon, 1937
that algebra is **circuitry**

By 1949, the inference is complete

Boole, 1854
reasoning is **algebra**

Shannon, 1937
that algebra is **circuitry**

Turing, 1936
one machine can run **any program**

McCulloch and Pitts, 1943
simplified neurons compute **logic**

By 1949, the inference is complete

Boole, 1854
reasoning is **algebra**

Shannon, 1937
that algebra is **circuitry**

Turing, 1936
one machine can run **any program**

McCulloch and Pitts, 1943
simplified neurons compute **logic**

KEY IDEA

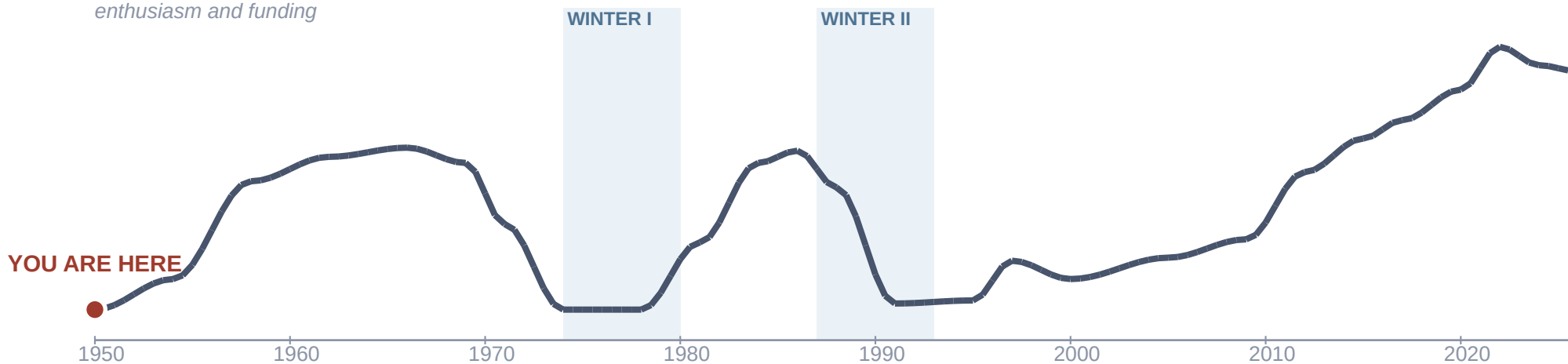
If brains compute, machines can, in principle, do what brains do. AI begins as an inference, not an invention.

PART 2

Turing's question

Philosophy becomes engineering · 1950

enthusiasm and funding



1950

Judge the cook by the food



“Can machines think?” → **too vague to answer.**

Replace it with a **test**: if sustained text conversation cannot tell machine from human, it passes. **Behaviour, measured.**

1950

Judge the cook by the food



“Can machines think?” → **too vague to answer.**

Replace it with a **test**: if sustained text conversation cannot tell machine from human, it passes. **Behaviour, measured.**

KEY IDEA

Deep legacy: AI becomes a field that advances by **benchmarks**. Remember this when a contest ends every argument in 2012.

He answered the objections in advance

“No feeling, no thinking.” → You cannot verify anyone’s inner life but your own. Extend machines the courtesy you extend people, when behaviour warrants.

He answered the objections in advance

“No feeling, no thinking.” → You cannot verify anyone’s inner life but your own. Extend machines the courtesy you extend people, when behaviour warrants.

“Only does what it is told.” → *“Machines take me by surprise with great frequency.”*

He answered the objections in advance

“No feeling, no thinking.” → You cannot verify anyone’s inner life but your own. Extend machines the courtesy you extend people, when behaviour warrants.

“Only does what it is told.” → *“Machines take me by surprise with great frequency.”*

“Gödel limits machines.” → What is the evidence **humans** are not comparably limited?

He answered the objections in advance

“No feeling, no thinking.” → You cannot verify anyone’s inner life but your own. Extend machines the courtesy you extend people, when behaviour warrants.

“Only does what it is told.” → *“Machines take me by surprise with great frequency.”*

“Gödel limits machines.” → What is the evidence **humans** are not comparably limited?

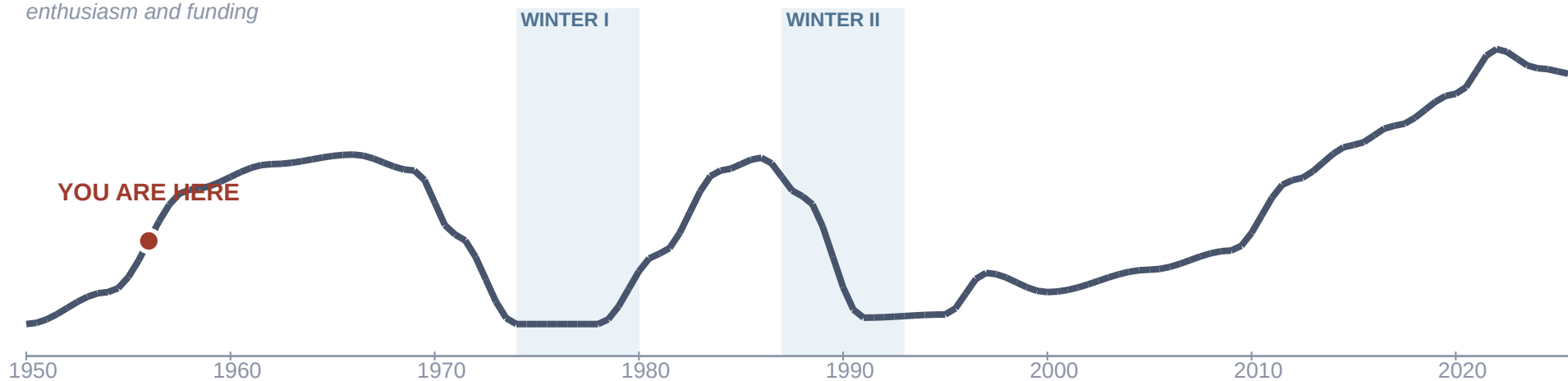
Short, witty, the field’s best primary source. Read it this week; every 2026 dinner-table argument is in it, with a courteous rebuttal.

PART 3

The christening

A name, a conjecture, two families · 1956

enthusiasm and funding



1956

Dartmouth, summer 1956



John McCarthy, 28, coins a name:
artificial intelligence.

Chosen partly to dodge the existing field (cybernetics) and its dominant personality.
Names are strategy.

1956

Dartmouth, summer 1956



John McCarthy, 28, coins a name:
artificial intelligence.

Chosen partly to dodge the existing field (cybernetics) and its dominant personality.
Names are strategy.

*“ Every aspect of learning... can **in principle** be so precisely described that a machine can be made to simulate it. “*

The Dartmouth proposal, 1955

A **conjecture, stated as a premise**: significant progress expected in one summer. Currently seventy years short.

The founders' forecasts

1957

Chess champion and a machine-discovered theorem **within ten years**. (Chess: forty years. Theorem: pending.)

The founders' forecasts

1957

Chess champion and a machine-discovered theorem **within ten years**. (Chess: forty years. Theorem: pending.)

1965

“Machines will be capable, within twenty years, of doing any work a man can do.”

The founders' forecasts

1957

Chess champion and a machine-discovered theorem **within ten years**. (Chess: forty years. Theorem: pending.)

1965

“Machines will be capable, within twenty years, of doing any work a man can do.”

KEY IDEA

Not fools: among the century's best scientists, fresh from real success. The lesson: **early success on toy versions is weak evidence about the full problem**. 1965 and 2026 rhyme.

Two rival families

Symbolic

rules written by hand

transparent, auditable

...someone must write the rules

Two rival families

Symbolic

rules written by hand

transparent, auditable

...someone must write the rules

Connectionist

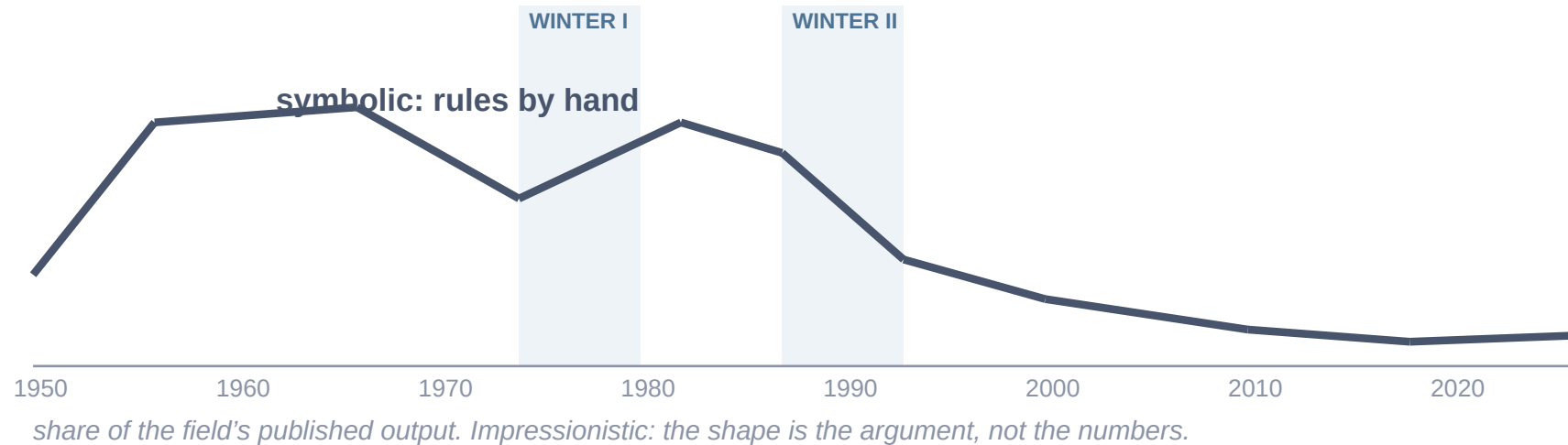
behaviour learned from data

learns what no one can write down

...opaque (Week 2's problem)

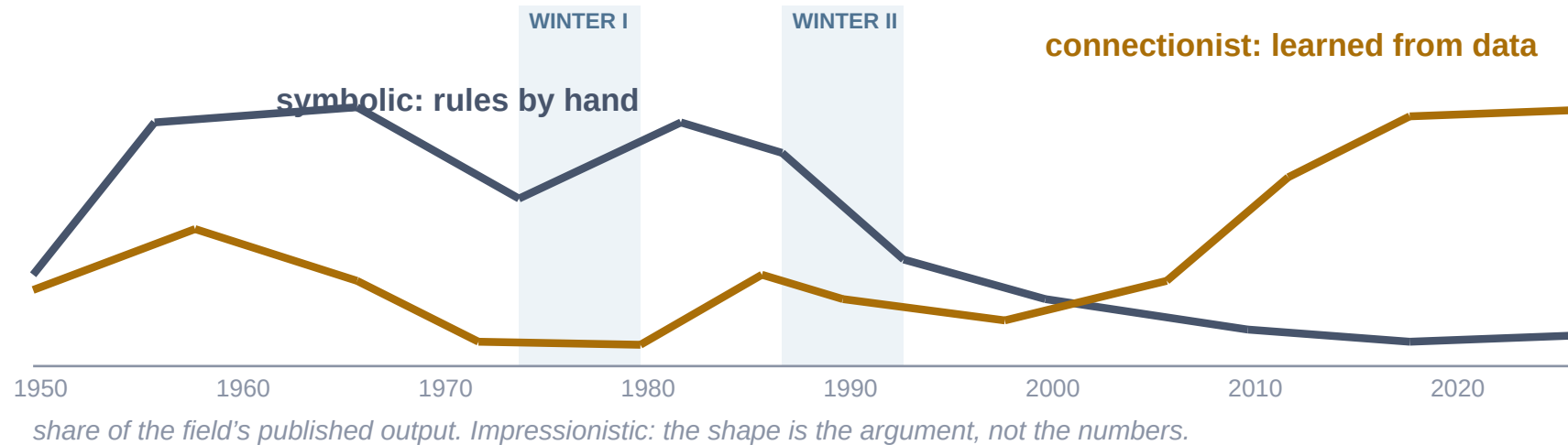
Symbolic dominates to the 1990s, crashes twice. Connectionism loses for thirty years, **then wins nearly everything**. Watch the frontier recombine them by the end of today.

Seventy years of pendulum



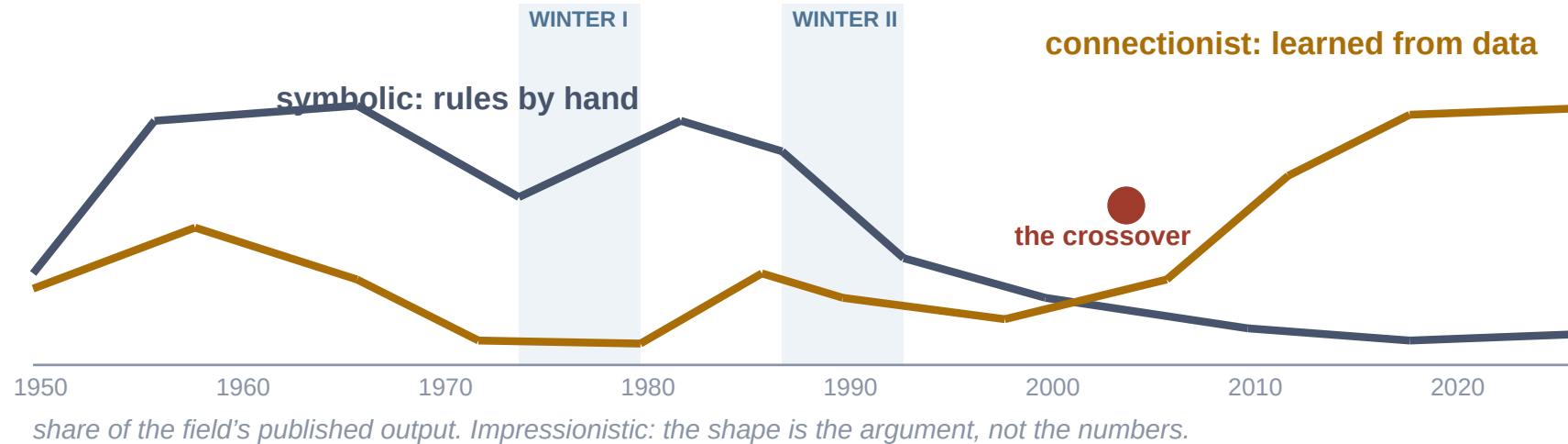
Symbolic dominates from Dartmouth into the 1990s. It also owns both crashes: the eras that overpromised were the eras that led.

Seventy years of pendulum



Connectionism spends thirty years as the losing side, quietly compounding through both winters, then takes nearly everything.

Seventy years of pendulum



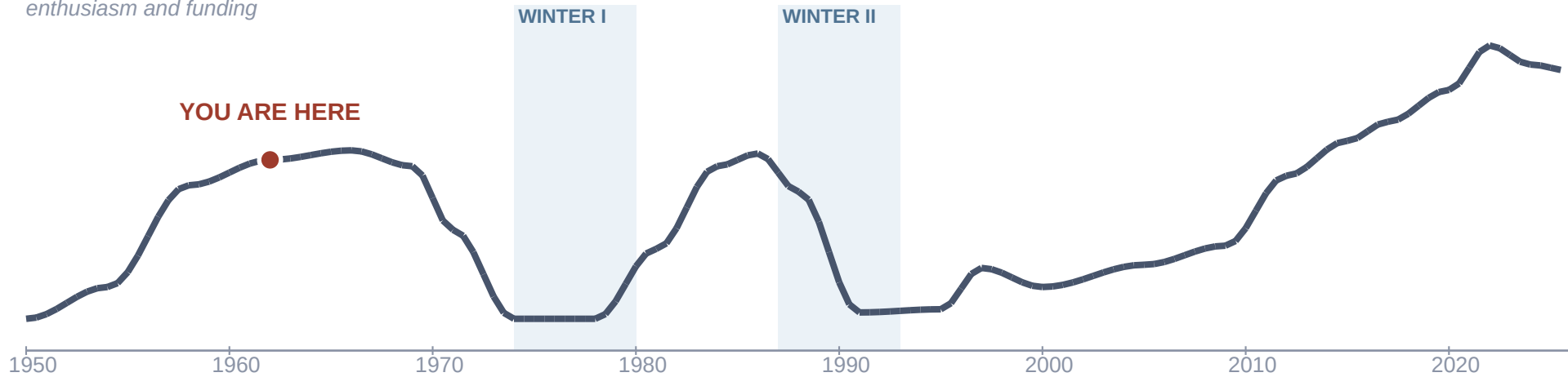
Note the crossover: **around 2005 to 2012**, before the public noticed. And note what the frontier is doing now: putting equations back into learning systems.

PART 4

The golden years

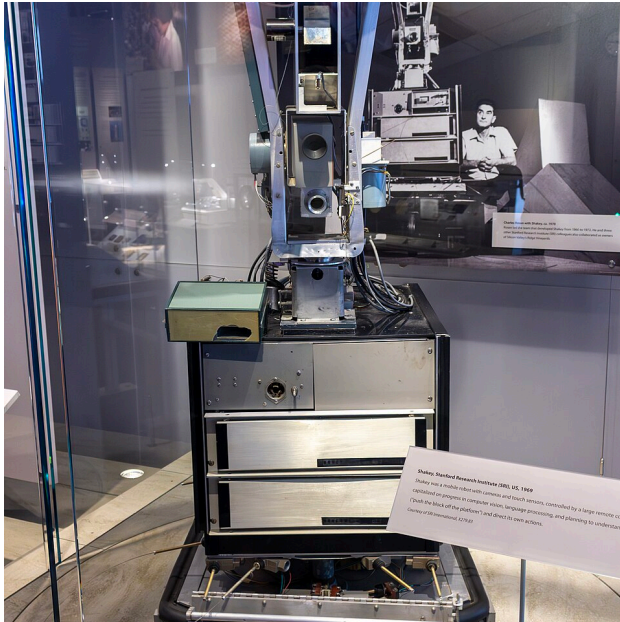
Everything works, in the shallow end · 1956 to 1969

enthusiasm and funding



1956 TO 1969

“The first electronic person”

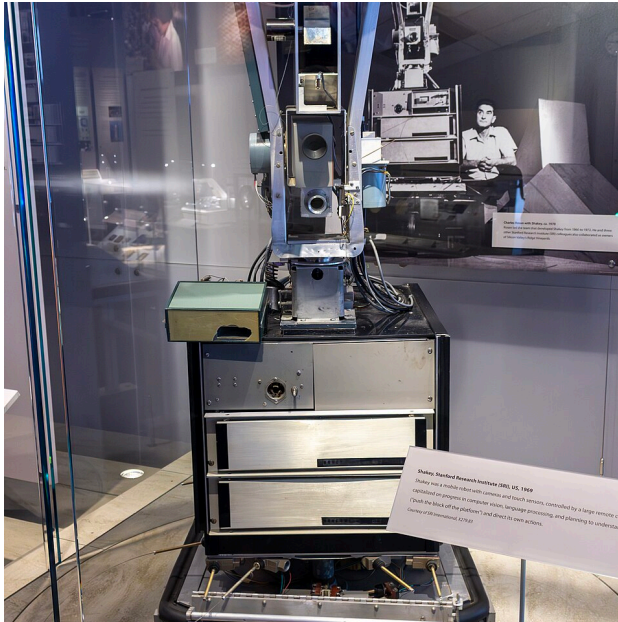


Theorems proved. LISP invented. SHRDLU converses about blocks. Shakey earns that magazine headline.

File the phrase for the next 2026 headline you read.

1956 TO 1969

“The first electronic person”



Theorems proved. LISP invented. SHRDLU converses about blocks. Shakey earns that magazine headline.

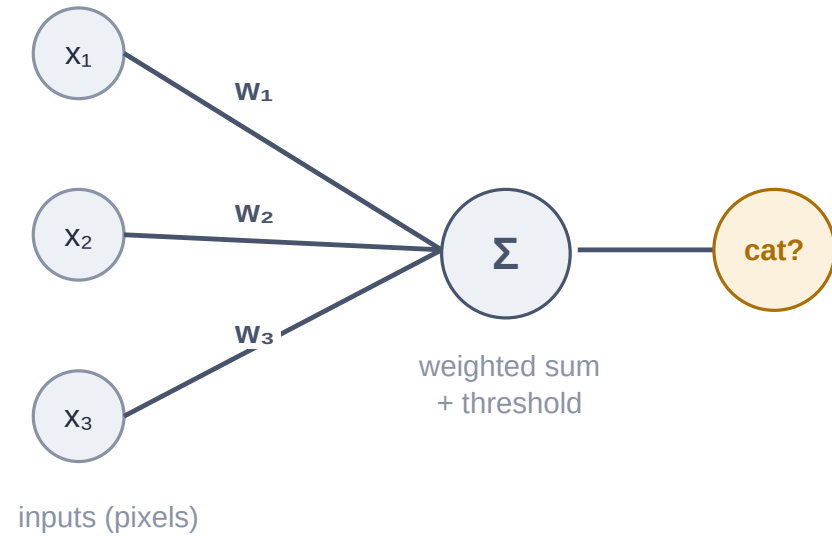
File the phrase for the next 2026 headline you read.

EVERYDAY ANALOGY

The method: **microworlds**. A swimming pool with the shallow end roped off. Everything learned there is true. **The ocean differs in kind, not degree.**

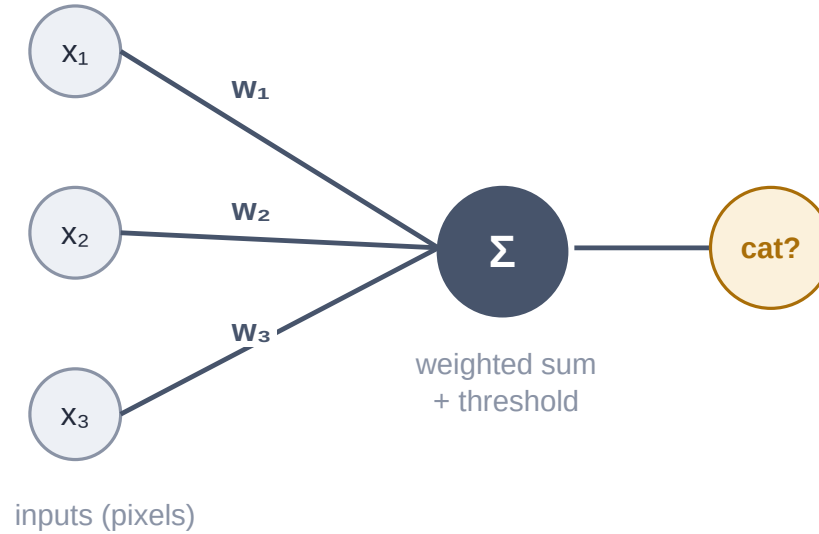
1958: the machine that learns

Each input gets a **dial** (a weight): push toward yes, toward no, or ignore.



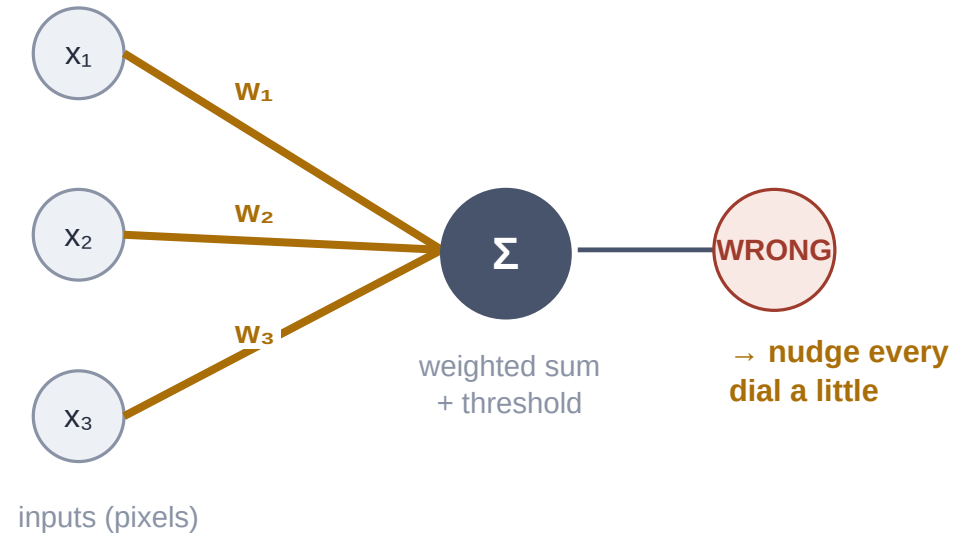
1958: the machine that learns

Sum, threshold, verdict.



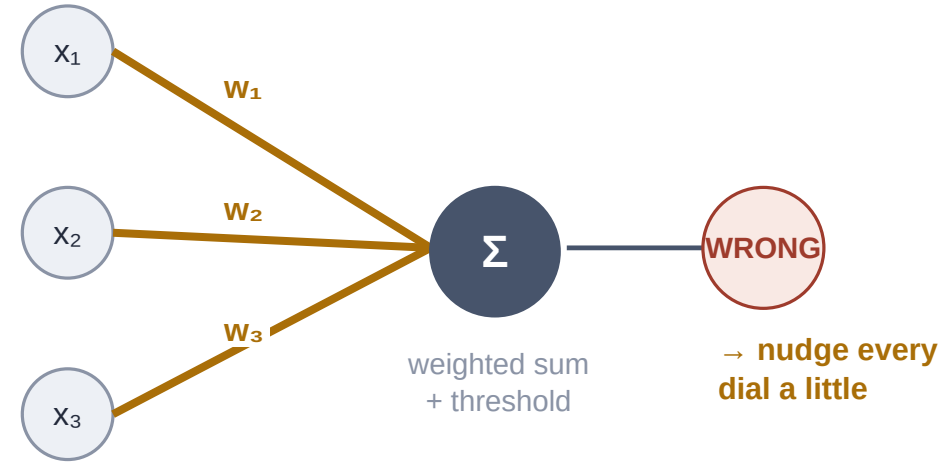
1958: the machine that learns

Wrong? **Nudge every dial** slightly toward less wrong. Right? Touch nothing.



1958: the machine that learns

Nobody ever tells it what a cat looks like.



inputs (pixels)

↻ repeat, thousands of times. The knowledge ends up in the dials.

The darts loop

EVERYDAY ANALOGY

You do not study projectile physics. You **throw, look, adjust your arm**, until the arm knows what the mind cannot write down.

The darts loop

EVERYDAY ANALOGY

You do not study projectile physics. You **throw, look, adjust your arm**, until the arm knows what the mind cannot write down.

KEY IDEA

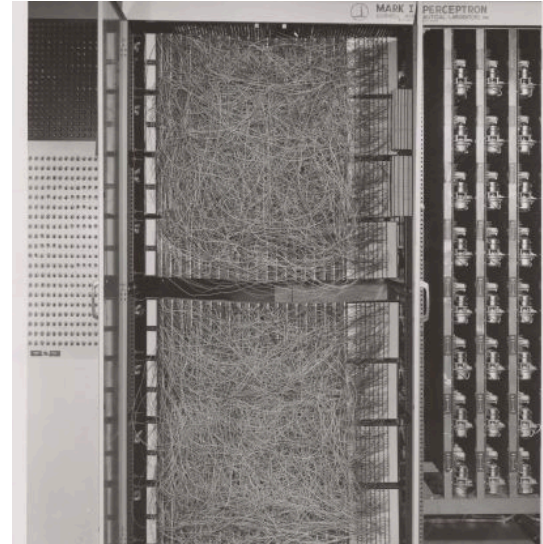
That loop, wrapped in a century of maths, **still trains the largest models on Earth**.
Week 3: gradient descent = which way to nudge; backpropagation = who gets the blame.

1958

What the papers said

400

pixels: the Mark I Perceptron's entire view of the
world

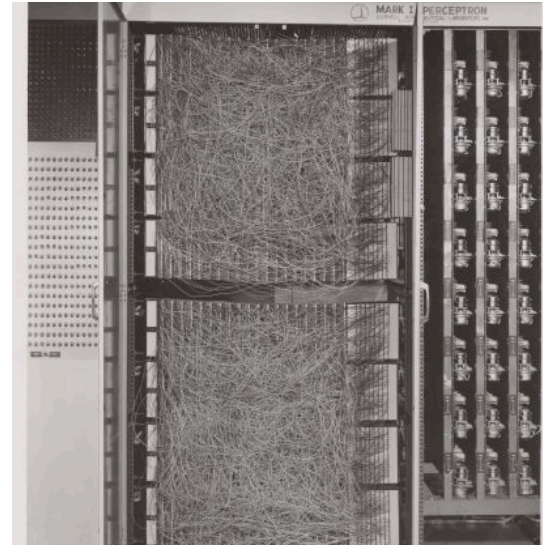


1958

What the papers said

400

pixels: the Mark I Perceptron's entire view of the world



*“ ...the embryo of an electronic computer that the Navy expects will be able to walk, talk, see, write, reproduce itself and be **conscious of its existence.** ”*

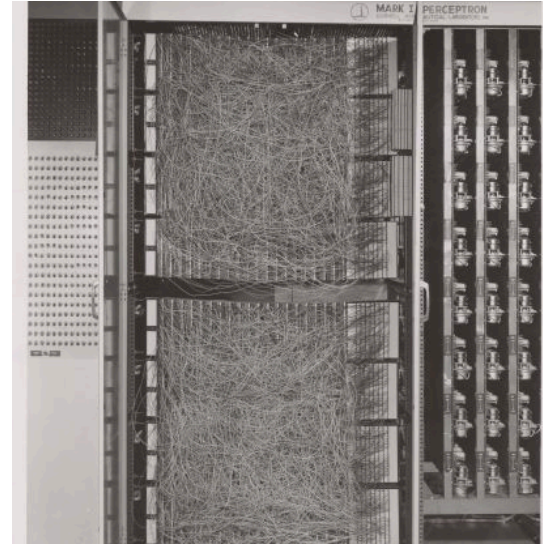
The New York Times, July 1958

1958

What the papers said

400

pixels: the Mark I Perceptron's entire view of the world



*“ ...the embryo of an electronic computer that the Navy expects will be able to walk, talk, see, write, reproduce itself and be **conscious of its existence**. ”*

The New York Times, July 1958

The gap, not the machine, is the exhibit. Printed two years into the field's existence.

I am sad about my job.

Why do you say you are sad about
your job?

A “therapist” by **keyword** → **template**.

THE FLUENCY TRAP

ELIZA, 1966

It understands **nothing**. Built to prove how hollow the trick was.

I am sad about my job.

Why do you say you are sad about your job?

My boss ignores me.

Tell me more about your boss.

The demonstration backfired: users confided at length. **Weizenbaum's own secretary asked him to leave the room.**

I am sad about my job.

Why do you say you are sad about your job?

My boss ignores me.

Tell me more about your boss.

keyword → template. Understanding: zero.

The **ELIZA effect**: we project understanding onto fluency. A fact about **us**, so it **compounds** as machines improve.

Standing rule: test on substance, never fluency.

I am sad about my job.

Why do you say you are sad about your job?

My boss ignores me.

Tell me more about your boss.

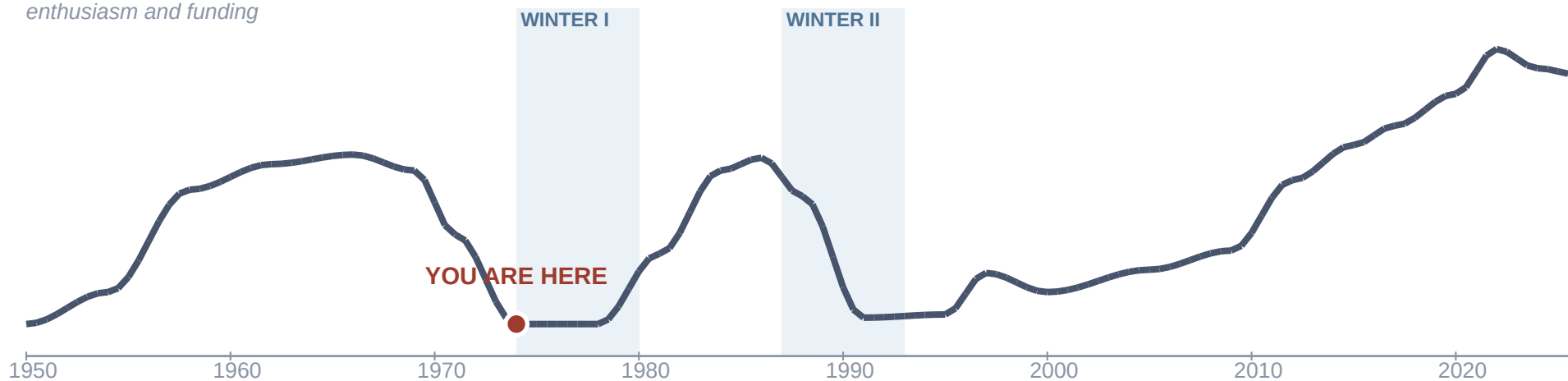
keyword → template. Understanding: zero.

PART 5

The first winter

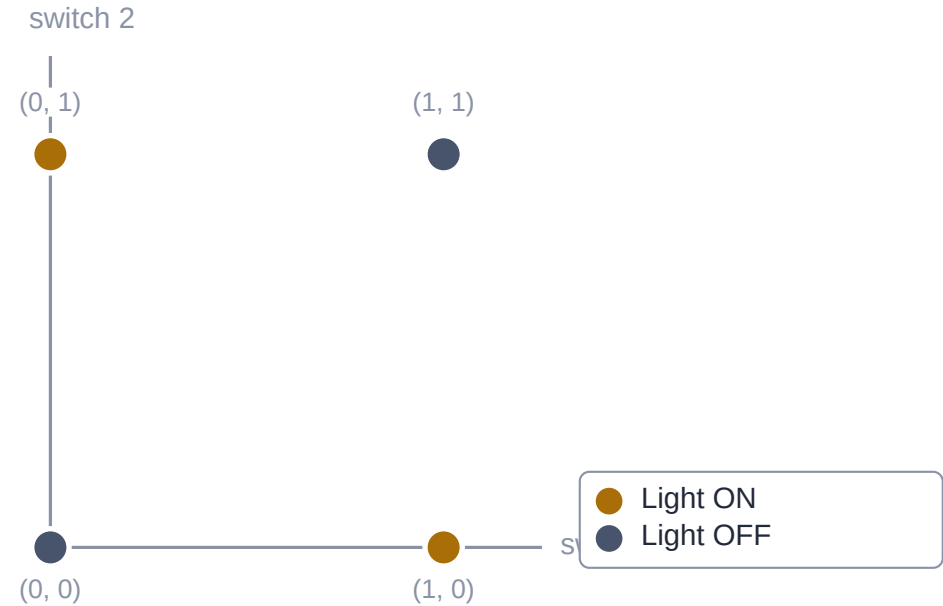
A correct theorem, read too broadly · 1969 to 1980

enthusiasm and funding



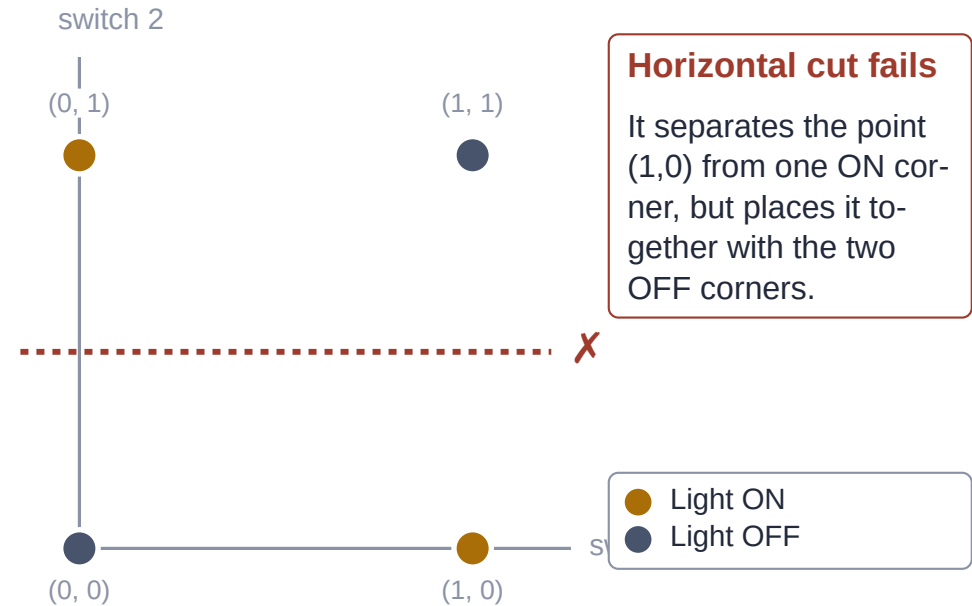
XOR: the stairwell light

Two switches, one light. ON when **exactly one** switch is up. Plot the four cases.



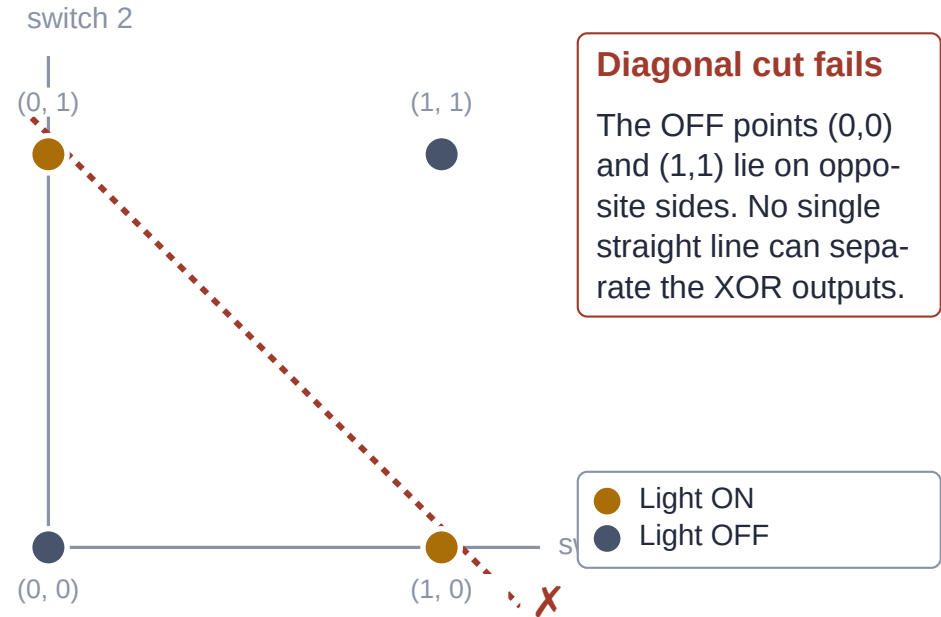
XOR: the stairwell light

A single-layer perceptron can only **draw one straight line**.



XOR: the stairwell light

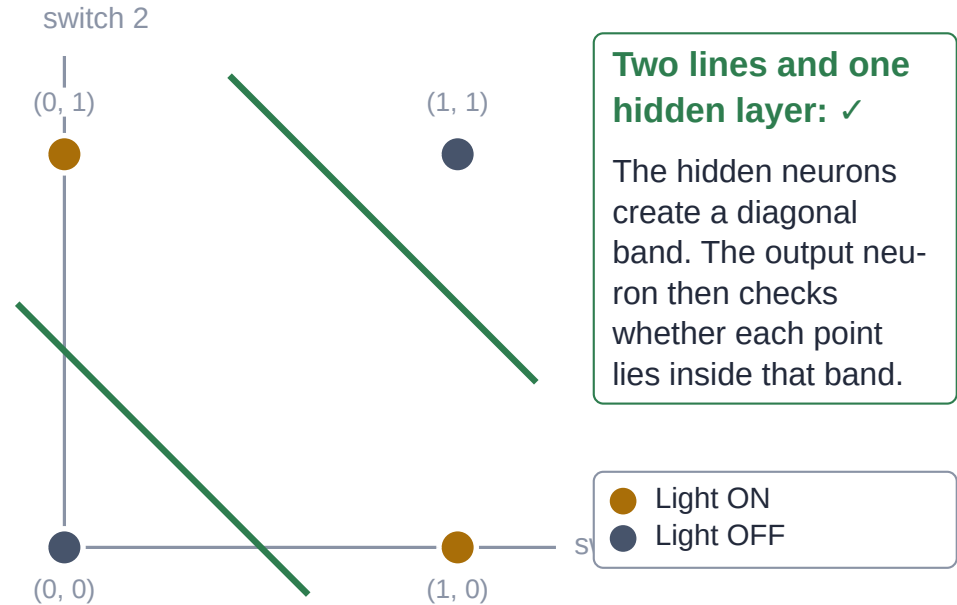
Diagonal corners. **Every line strands someone.** Proven impossible.



1969

XOR: the stairwell light

The fix (a hidden layer) was **known in 1969**. How to **train** it was not: solved 1986. **Seventeen years.**



1969 TO 1974

A theorem becomes a winter

EVERYDAY ANALOGY

A rigorous proof that a **bicycle** cannot climb stairs... and the world concludes **wheels are refuted**.

1969 TO 1974

A theorem becomes a winter

EVERYDAY ANALOGY

A rigorous proof that a **bicycle** cannot climb stairs... and the world concludes **wheels are refuted**.

KEY IDEA

Permanent discipline for every AI criticism you will ever read: **which version does this reach, and is there an escape route?** Sober twin: “route exists” is not “route is easy”.

The honest villain

10^4

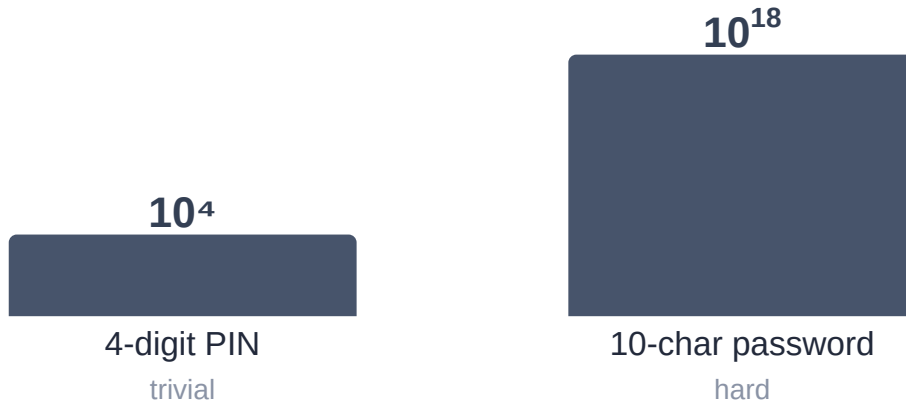


4-digit PIN

trivial

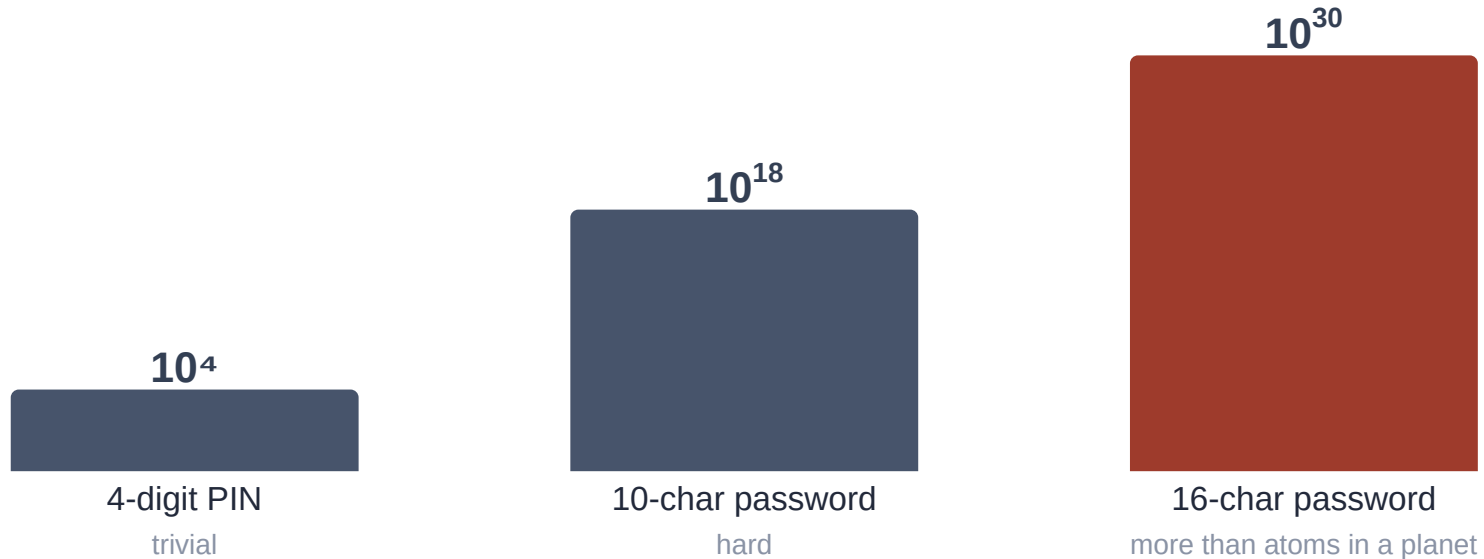
Search worked in microworlds because possibilities were few.

The honest villain



Each added element **multiplies** the branches.

The honest villain



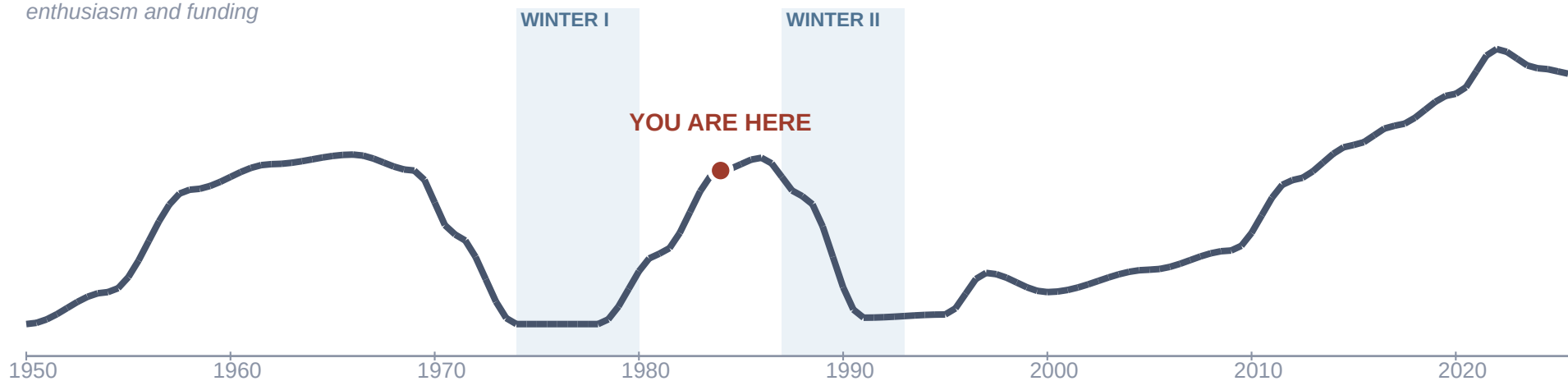
Past a small threshold, exponential is not “harder”. It is a wall. This module’s most important phrase: *“it does not scale.”*

PART 6

The comeback

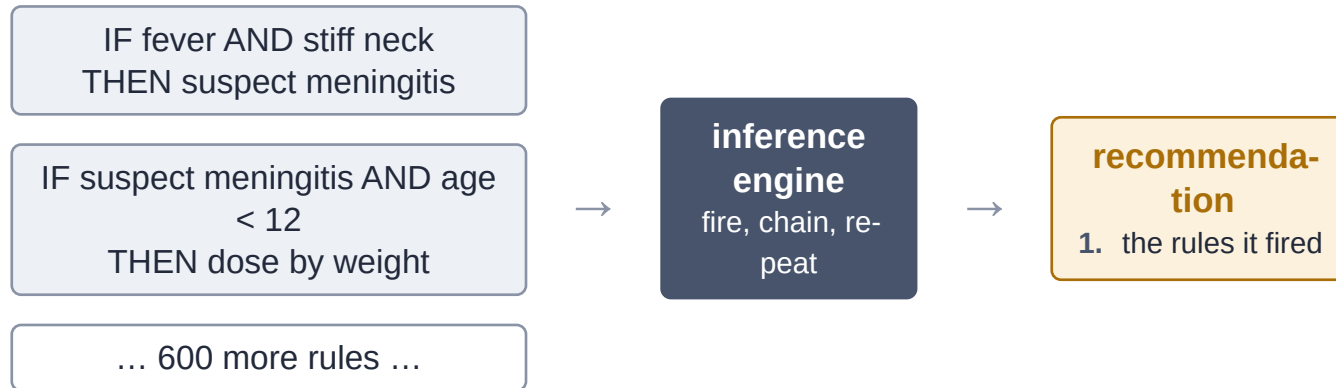
Sell the rules · 1980 to 1987

enthusiasm and funding



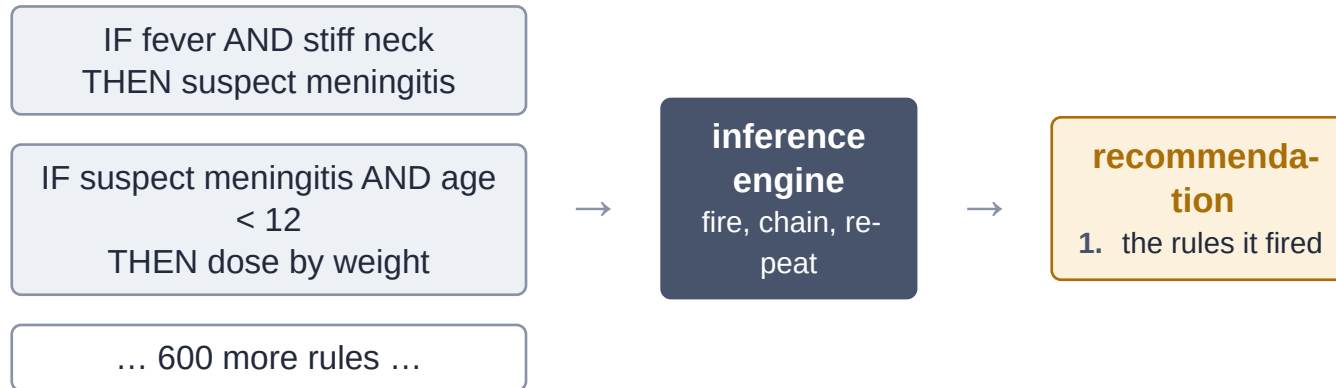
A smaller, sellable promise

One expert. One narrow domain. **Thousands of hand-written IF-THEN rules.**



A smaller, sellable promise

One expert. One narrow domain. **Thousands of hand-written IF-THEN rules.**



Symbolic AI's purest commercial form, and it can **show its work**: transparency Week 2 will make you miss.

1980S

The wins were real

≈ 40M



dollars saved per year: XCON configuring DEC's
orders

MYCIN matched specialist physicians in formal
evaluation

1980S

The wins were real

≈ 40M



dollars saved per year: XCON configuring DEC's
orders

MYCIN matched specialist physicians in formal
evaluation

Corporate AI groups, venture capital, Japan's national programme, a bespoke **Lisp machine** hardware industry. (And MYCIN was never deployed: liability, workflow, trust. The **sociotechnical** gap: Week 2 in one anecdote.)

1987

Why it died: three diseases

1 · Brittleness : one inch outside the foreseen, absurd **confident** failure. The phrasebook tourist.

Why it died: three diseases

1 · Brittleness : one inch outside the foreseen, absurd **confident** failure. **The phrasebook tourist.**

2 · Knowledge bottleneck : experts cannot tell you what they know. **Grandmother's salt:** "until it tastes right."

Why it died: three diseases

1 · Brittleness : one inch outside the foreseen, absurd **confident** failure. **The phrasebook tourist.**

2 · Knowledge bottleneck : experts cannot tell you what they know. **Grandmother's salt:** "until it tastes right."

3 · Maintenance : rule 3,001 contradicts rule 214; bases rot. Cost grows with the **square** of ambition.

Why it died: three diseases

1 · Brittleness : one inch outside the foreseen, absurd **confident** failure. **The phrasebook tourist.**

2 · Knowledge bottleneck : experts cannot tell you what they know. **Grandmother's salt:** “until it tastes right.”

3 · Maintenance : rule 3,001 contradicts rule 214; bases rot. Cost grows with the **square** of ambition.

KEY IDEA

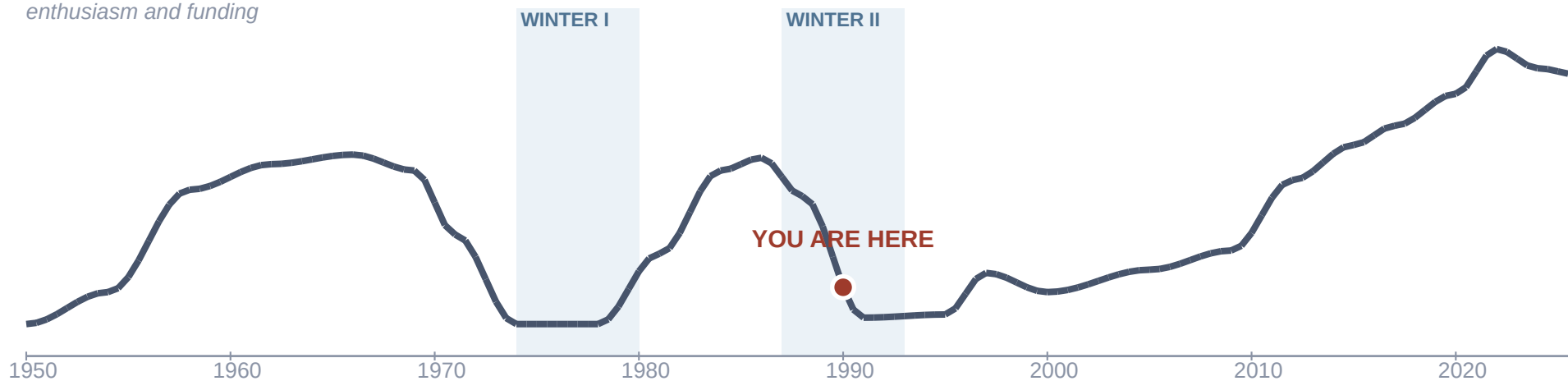
Machine learning's wager: **tacit knowledge can be absorbed from examples** instead of extracted by interview. Ten thousand labelled stews, and it learns the judgement, exactly as grandmother did.

PART 7

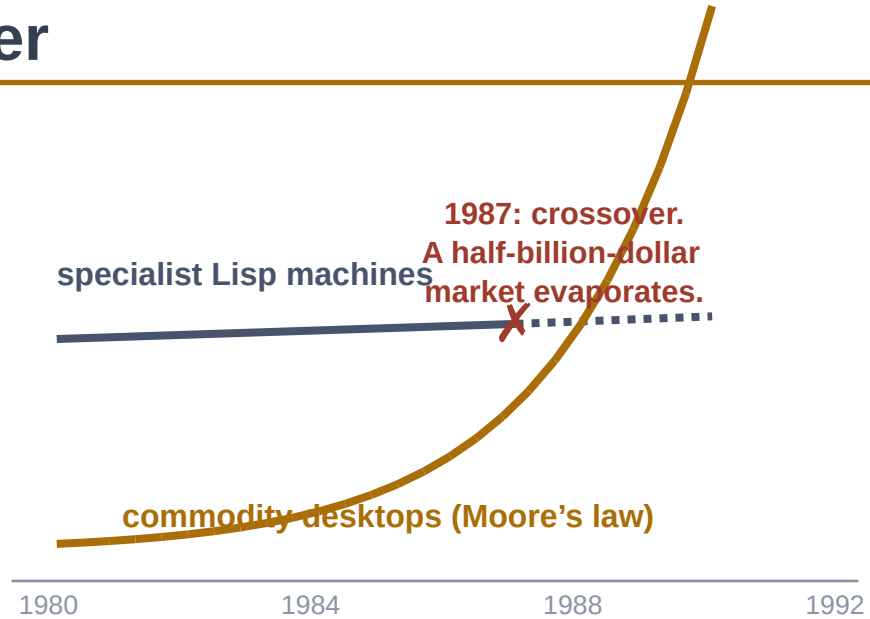
The second winter

An industry collapses; a field renames itself · 1987 to 1993

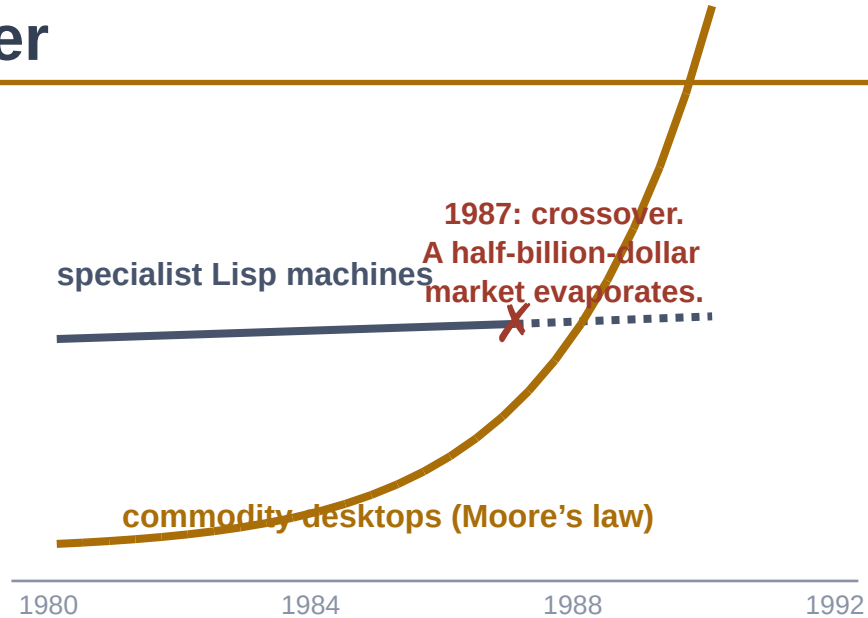
enthusiasm and funding



1987: the crossover

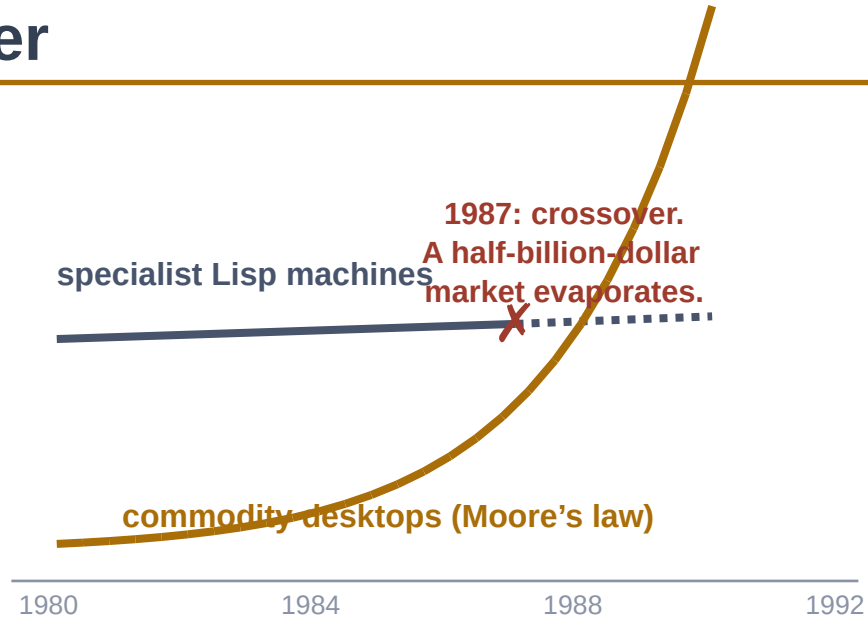


1987: the crossover



Software followed hardware: the three diseases presented their invoices. “*Artificial intelligence*” became toxic to funders.

1987: the crossover



Software followed hardware: the three diseases presented their invoices. “*Artificial intelligence*” became toxic to funders.

KEY IDEA

Researchers kept the work, changed the label: **machine learning**. Camouflage, and **honest signalling**: the field really had shifted, from writing rules to learning from data.

THROUGH BOTH WINTERS

The keepers of the flame



Hinton · LeCun · Bengio worked on neural networks when it was career poison. Canadian public money kept the flame lit.

The keepers of the flame

Hinton · LeCun · Bengio worked on neural networks when it was career poison. Canadian public money kept the flame lit.

Backprop popularised 1986 (XOR's escape, seventeen years late). LeCun's networks read a meaningful share of US cheques **in the 1990s**: deep learning in production a decade before it was famous. The trio: 2018 Turing Award; Hinton, later, a Nobel.

The keepers of the flame

Hinton · LeCun · Bengio worked on neural networks when it was career poison. Canadian public money kept the flame lit.

Backprop popularised 1986 (XOR's escape, seventeen years late). LeCun's networks read a meaningful share of US cheques **in the 1990s**: deep learning in production a decade before it was famous. The trio: 2018 Turing Award; Hinton, later, a Nobel.

KEY IDEA

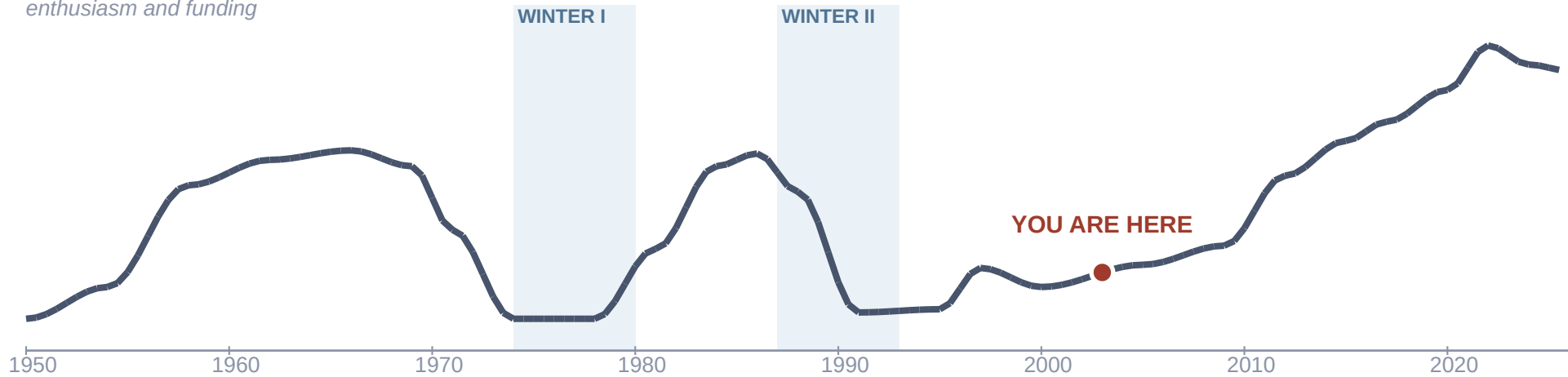
A dormant field and a dead field look identical from outside. Winters punish popularity, not truth.

PART 8

The quiet revolution

AI colonises daily life, under other names · 1990s to 2011

enthusiasm and funding



1990S TO 2011

You used “dead” AI every day

Spam filter

learned from millions of
flags

Credit scoring

patterns, not rules

Fraud detection

which purchases are not
you

Search ranking

learned relevance

1990S TO 2011

You used “dead” AI every day

Spam filter

learned from millions of
flags

Credit scoring

patterns, not rules

Fraud detection

which purchases are not
you

Search ranking

learned relevance

KEY IDEA

The pivot of the whole history: **rules** → **data** · **logic** → **probability** · **demos** → **benchmarks**.
By 2005 you met learned models daily while believing AI had failed.

THE OLD PARADIGM'S FINEST HOUR

Deep Blue, 1997



200,000,000

positions per second, against the world champion



200,000,000

positions per second, against the world champion

Brute search + hand-tuned rules, perfected, in a **closed** world. It beat Kasparov... and could not recognise a face, parse a sentence, or **transfer anything**. (Its most feared “genius” move: later explained as likely a **bug**.)



200,000,000

positions per second, against the world champion

Brute search + hand-tuned rules, perfected, in a **closed** world. It beat Kasparov... and could not recognise a face, parse a sentence, or **transfer anything**. (Its most feared “genius” move: later explained as likely a **bug**.)

KEY IDEA

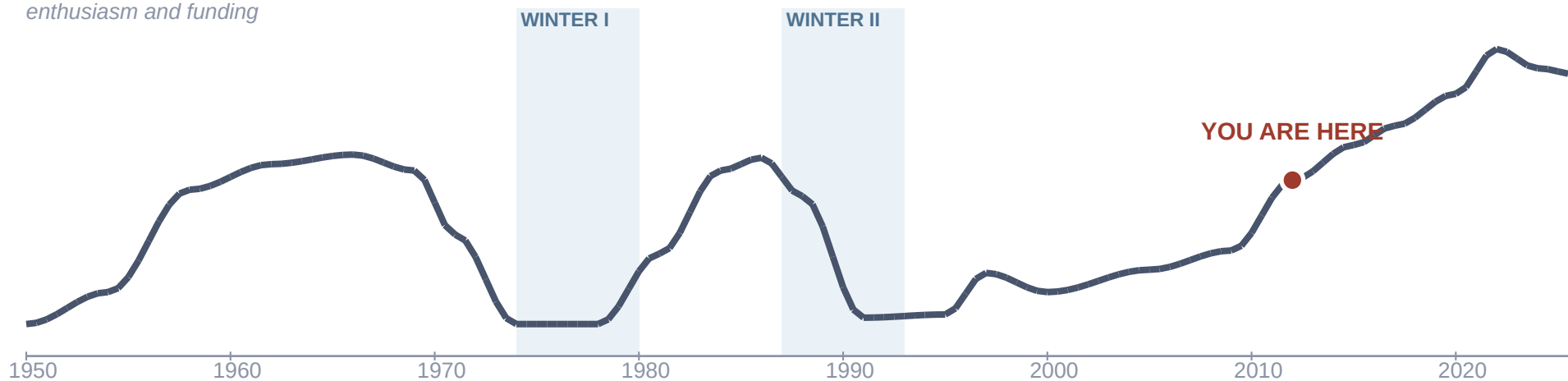
Days later: “*just brute force, not **real** intelligence.*” The **AI effect**: each milestone redefined away. Applied relentlessly, “real AI” = **whatever machines cannot yet do**.

PART 9

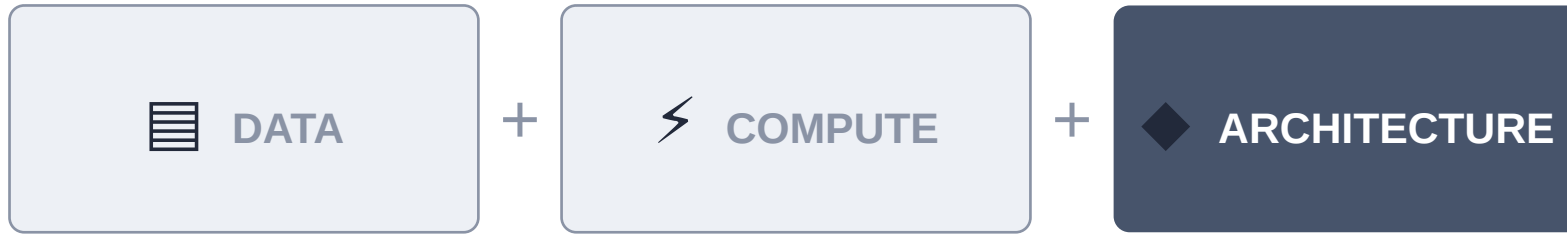
The big bang

Three supplies finally coexist · 2012

enthusiasm and funding



Three ingredients, one recipe



1958 · Perceptron starved. Right idea, 400 pixels.

Every **partial** combination was tried. Each has a name. Each stalled.

Three ingredients, one recipe



1997 · Deep Blue power, no transfer.

Every **partial** combination was tried. Each has a name. Each stalled.

Three ingredients, one recipe



2010 · ImageNet, before an arena with no champion.

Every **partial** combination was tried. Each has a name. Each stalled.

Three ingredients, one recipe

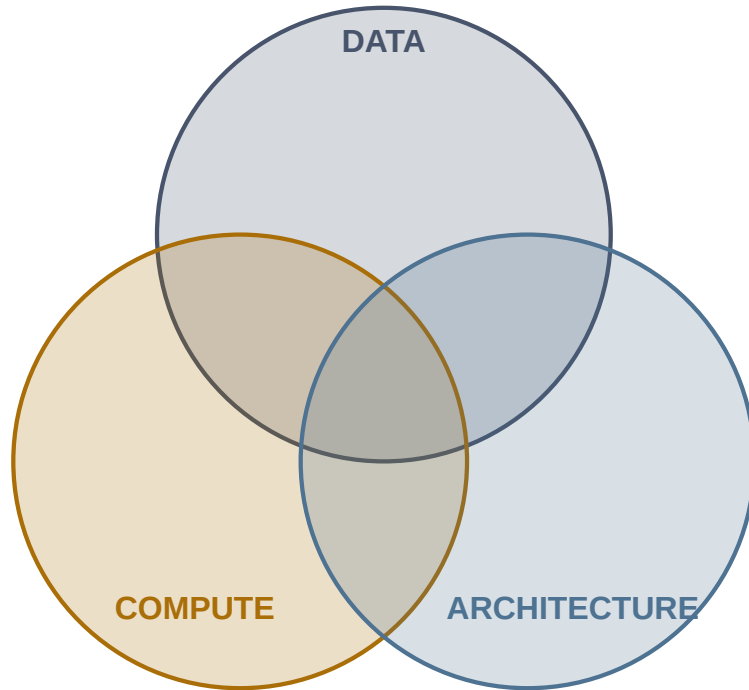


2012 · AlexNet all three, for the first time in history.

KEY IDEA

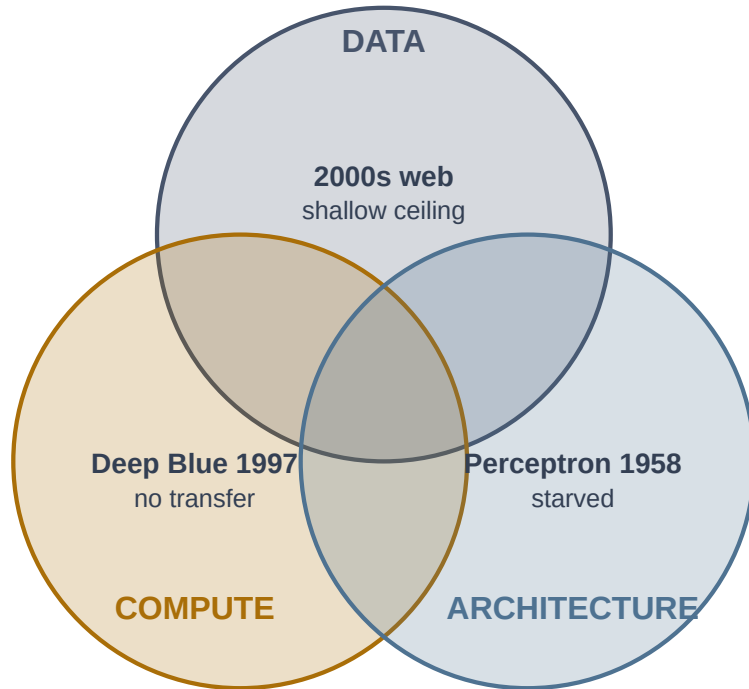
Deep learning ignites only when **data + compute + architecture** coexist. **A coincidence of supply; almost no new ideas.**

Every region has a name and a date



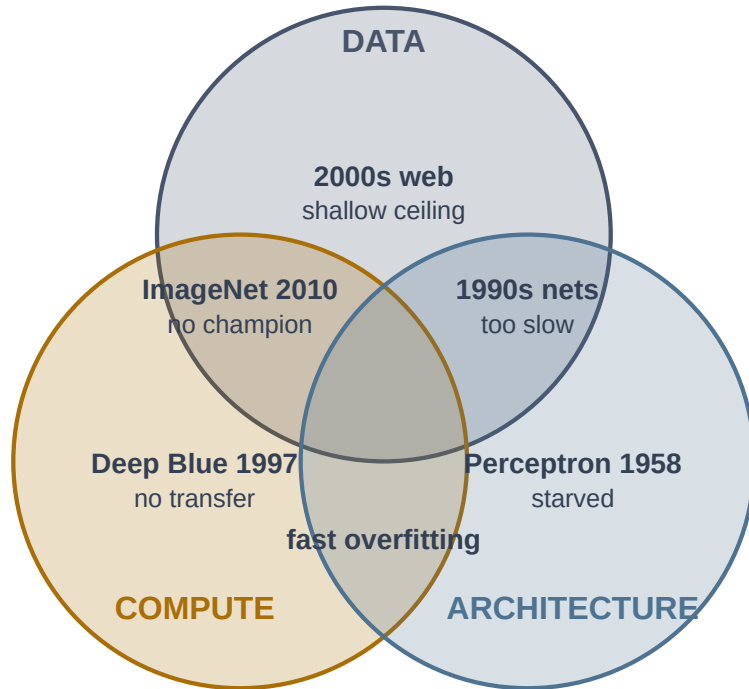
Three supplies. Deep learning needs all three **at once**.

Every region has a name and a date



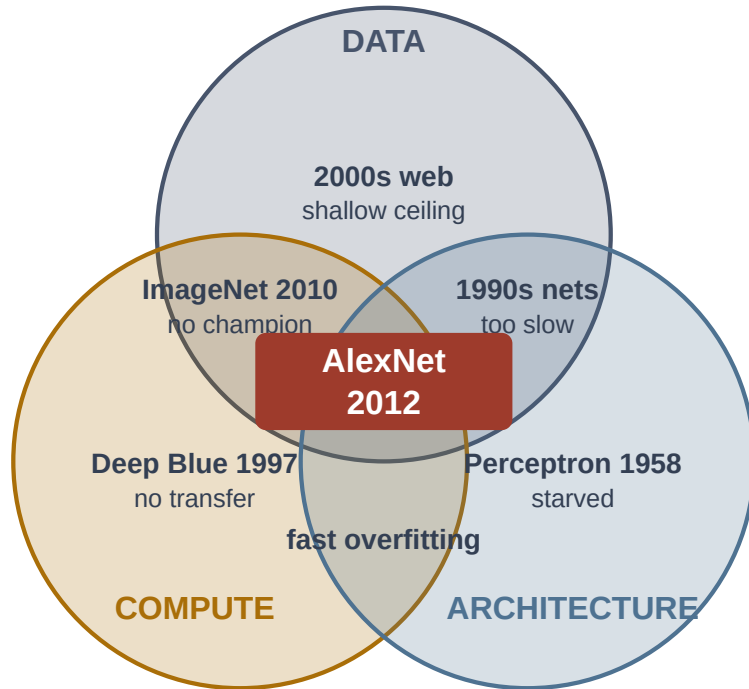
Each supply **on its own** has an era attached, and each one stalled.

Every region has a name and a date



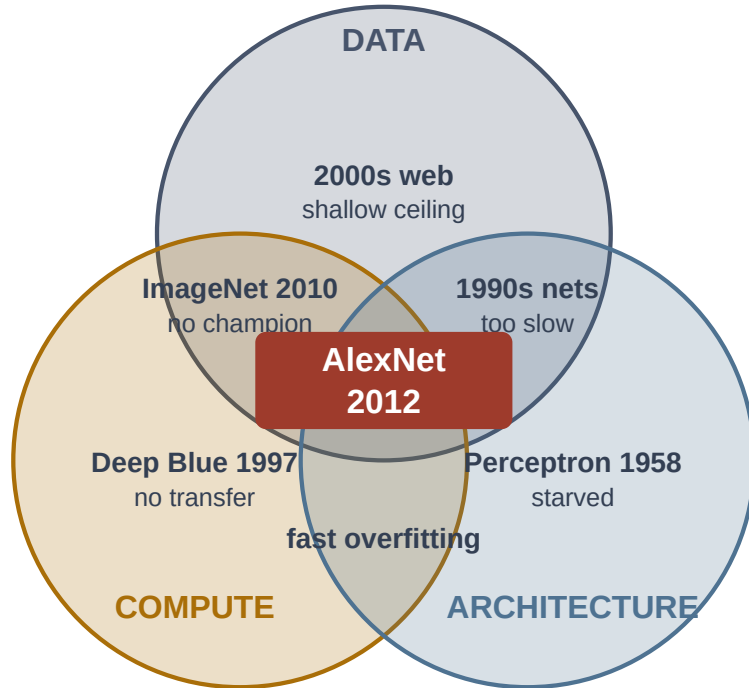
Each **pair** also has an era, and those stalled too. 2010 is the painful one: the data and the machines were both there.

Every region has a name and a date



One region has a field in it.

Every region has a name and a date



Every region has a **name** and a **date**. Only the centre has a **field**.

Almost no new ideas. A coincidence of supply.

Whose spending built the road?



Compute: ten thousand adequate workers, built over two decades **for video games. Gamers paid for AI's infrastructure.**

Whose spending built the road?



Compute: ten thousand adequate workers, built over two decades **for video games. Gamers paid for AI's infrastructure.**

Data: Fei-Fei Li's ImageNet, 14M labelled photos, built against advice (*"service work, not science"*), plus a public annual contest: **the arena.**

Whose spending built the road?



Compute: ten thousand adequate workers, built over two decades **for video games. Gamers paid for AI's infrastructure.**

Data: Fei-Fei Li's ImageNet, 14M labelled photos, built against advice ("*service work, not science*"), plus a public annual contest: **the arena**. **Architecture:** waiting in a drawer since 1986 (backprop) and the 1990s (networks already reading cheques).

Whose spending built the road?



Compute: ten thousand adequate workers, built over two decades **for video games. Gamers paid for AI's infrastructure.**

Data: Fei-Fei Li's ImageNet, 14M labelled photos, built against advice (*"service work, not science"*), plus a public annual contest: **the arena**. **Architecture:** waiting in a drawer since 1986 (backprop) and the 1990s (networks already reading cheques).

EVERYDAY ANALOGY

The **adjacent industry subsidy**: railways built for coal end up carrying passengers. Ask always: **whose spending is building the road right now?**

Why the graphics card, and not the processor

CPU

a few brilliant workers



8 cores, each able to do
anything, one thing at a time

Why the graphics card, and not the processor

CPU

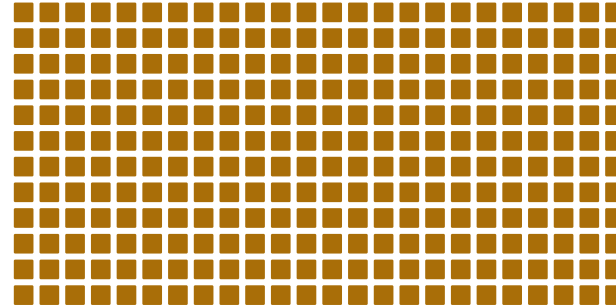
a few brilliant workers



8 cores, each able to do
anything, one thing at a time

GPU

ten thousand adequate ones



thousands of cores, each able to do
one simple sum, all at once

Why the graphics card, and not the processor

CPU

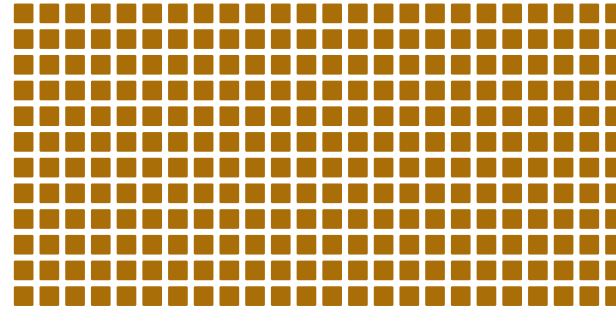
a few brilliant workers



8 cores, each able to do
anything, one thing at a time

GPU

ten thousand adequate ones



thousands of cores, each able to do
one simple sum. all at once

Training a network **is** millions of small multiplications with no argument about the order. Built for video games. **Gamers paid for AI's infrastructure.**

September 2012: the scoreboard argues

26%

2011 winner's error



15%

AlexNet, 2012

September 2012: the scoreboard argues

26%

2011 winner's error

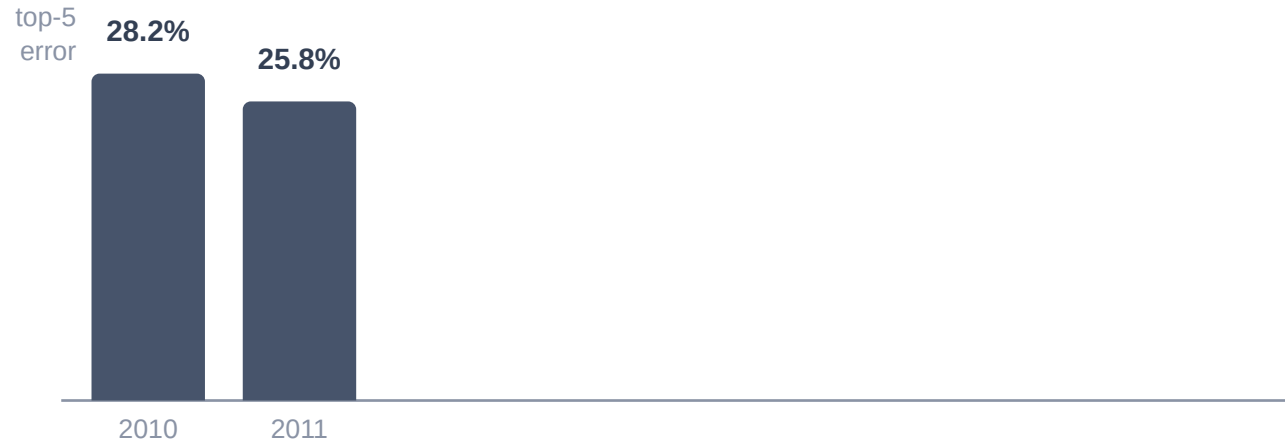


15%

AlexNet, 2012

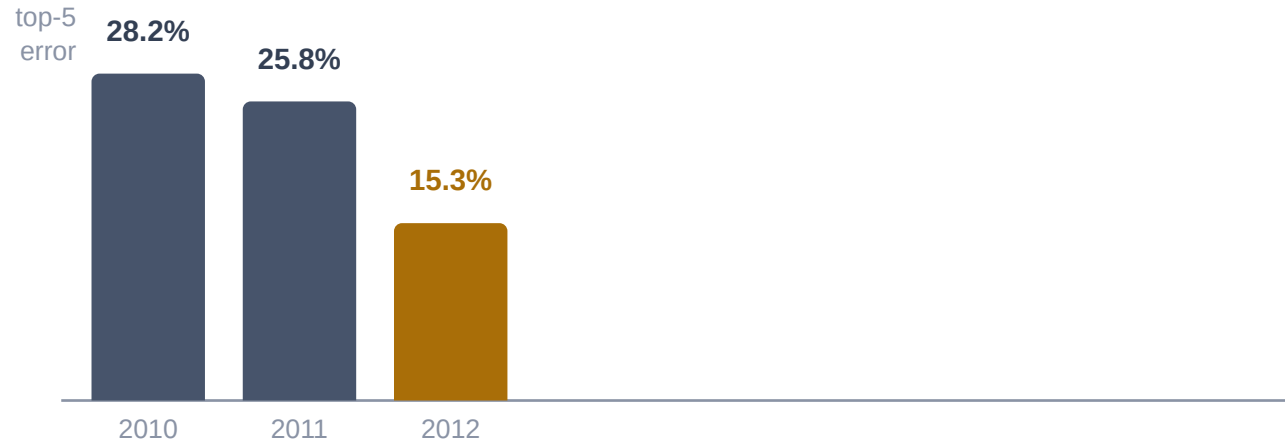
Two students, one professor, a winter-funded Canadian lab, **two consumer gaming GPUs in a desktop**. Contests this mature move by fractions of a point; AlexNet moved ten. Within one cycle, everyone was deep. Because the arena was public and scored, **there was nothing to argue about**.

What happened next, on the same benchmark



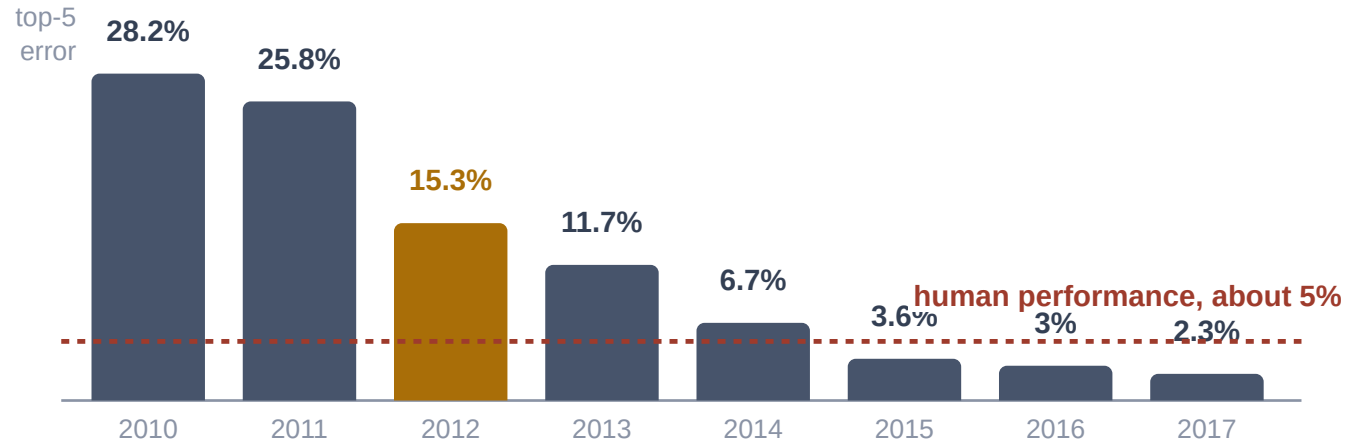
Two years of careful, hand-engineered progress: 28.2% to 25.8%.

What happened next, on the same benchmark

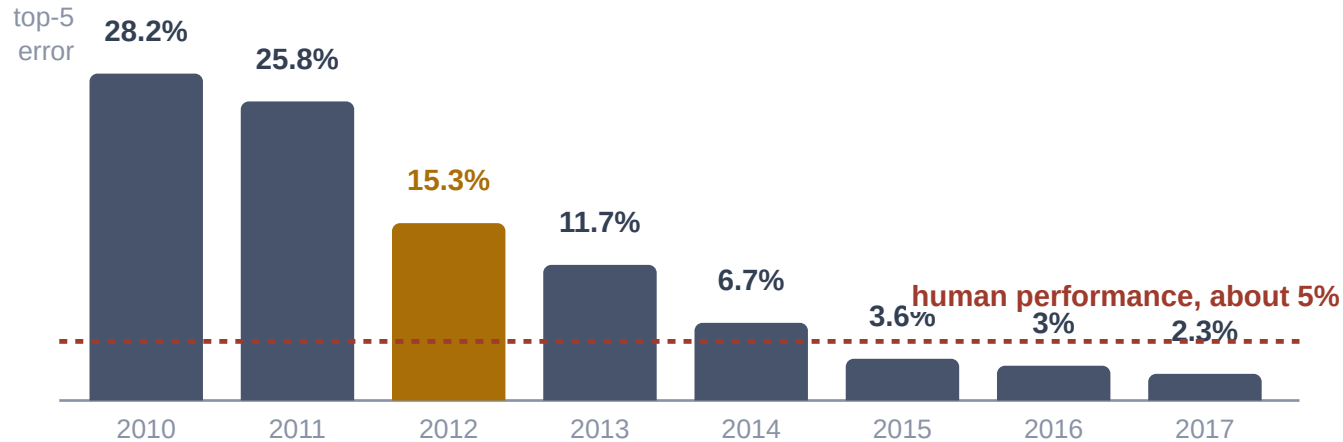


Then 2012.

What happened next, on the same benchmark



What happened next, on the same benchmark



Contests this mature move by fractions of a point. AlexNet moved **ten**, and within one cycle every serious entry was deep. Then the whole field walked past the human line and kept going.

THE PARADIGM SHIFT, SCORED

AlphaGo, 2016



10^{170}

Go positions

10^{80}

atoms in the observable universe

THE PARADIGM SHIFT, SCORED

AlphaGo, 2016



10^{170}

Go positions

10^{80}

atoms in the observable universe

Brute force **permanently out**; Go judgement is **tacit**. AlphaGo **learned** it: 30M human positions, then **self-play** beyond its teachers. 4-1 over Lee Sedol. **Move 37** broke centuries of principle. **Lovelace, meet the counterexample.**

THE PARADIGM SHIFT, SCORED

AlphaGo, 2016



10^{170}

Go positions

10^{80}

atoms in the observable universe

Brute force **permanently out**; Go judgement is **tacit**. AlphaGo **learned** it: 30M human positions, then **self-play** beyond its teachers. 4-1 over Lee Sedol. **Move 37** broke centuries of principle. **Lovelace, meet the counterexample.**

KEY IDEA

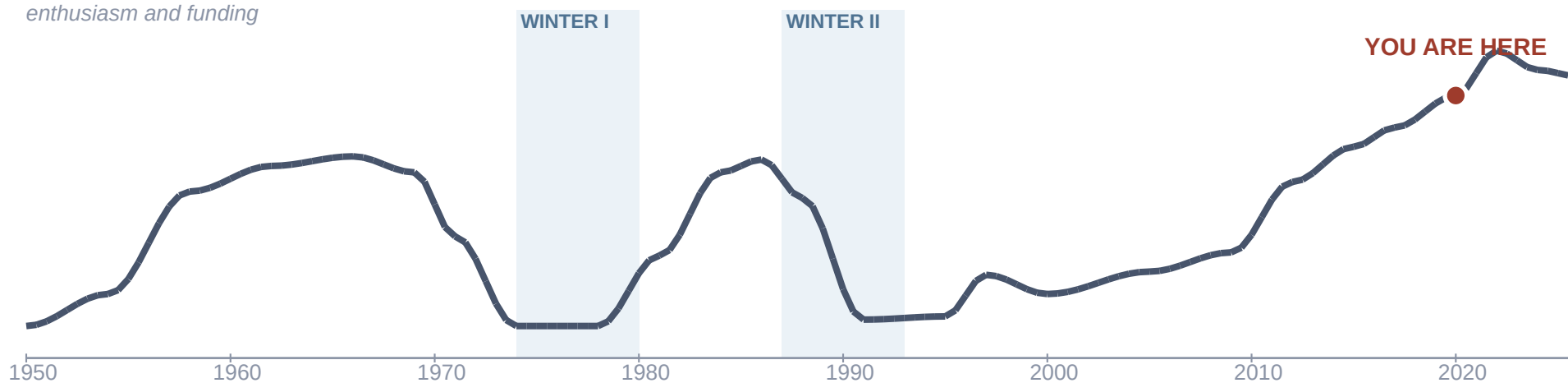
Exam contrast: Deep Blue = rules + brute search, closed world, zero transfer. AlphaGo = **learned evaluation + self-play**, and a successor learning three games with one method. (Lee's "divine" move 78 beat the machine once. Same board, same week.)

PART 10

The language era

Attention, scale, and a chat box · 2017 to 2023

enthusiasm and funding



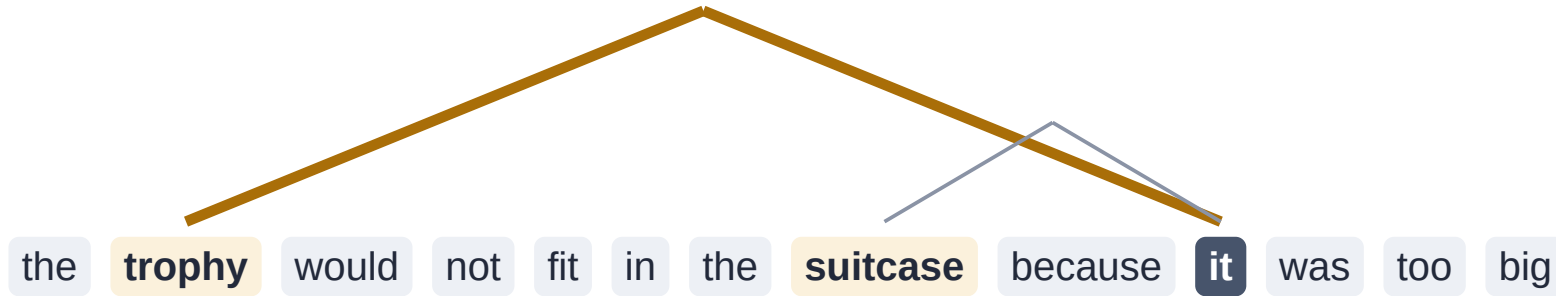
Attention: the whole book open on the desk

Before 2017: reading through a **keyhole**, one word at a time, strictly sequential.

the trophy would not fit in the suitcase because it was too big

Attention: the whole book open on the desk

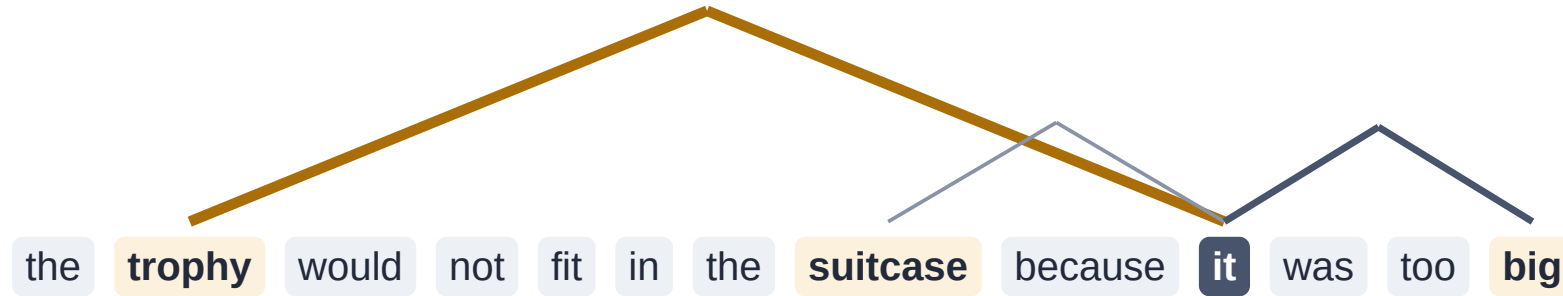
The transformer: every word looks at **every other word simultaneously**, and **learns how much each matters**.



What does “it” refer to? Attention learns to weigh **trophy** vs **suitcase**...

Attention: the whole book open on the desk

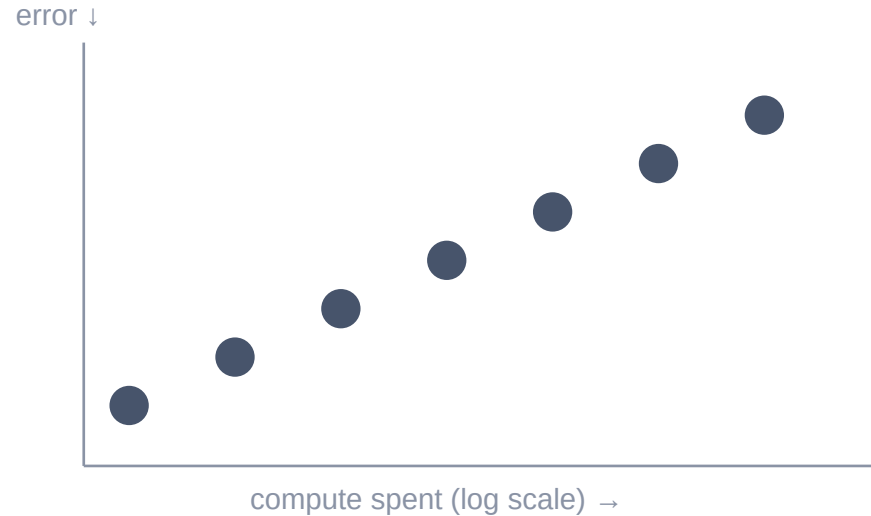
The transformer: every word looks at **every other word simultaneously**, and **learns how much each matters**.



What does “it” refer to? ...and “big” casts the deciding vote: big things fail to fit. So: the trophy.

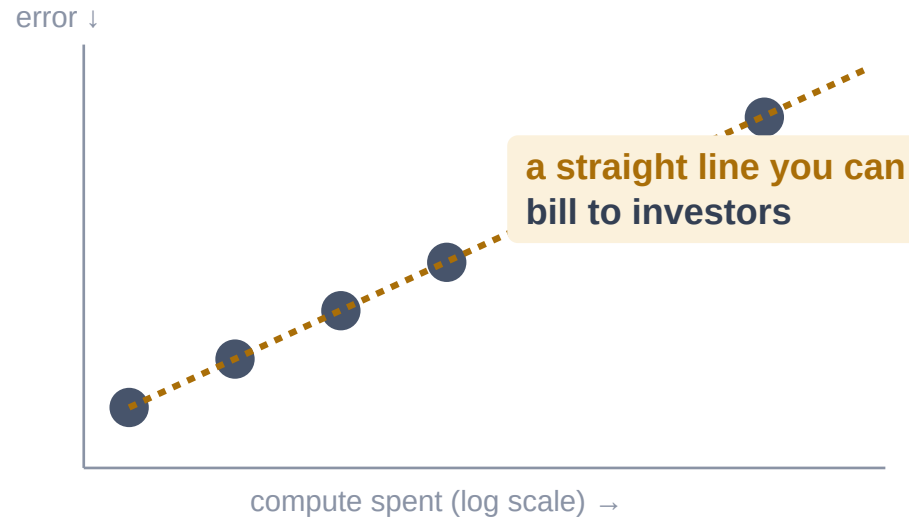
Parallel arithmetic: **the architecture the GPU was waiting for**. Hidden bill: cost grows with the **square** of length.
Week 8 exists to dodge it.

Scaling laws: progress becomes purchasable



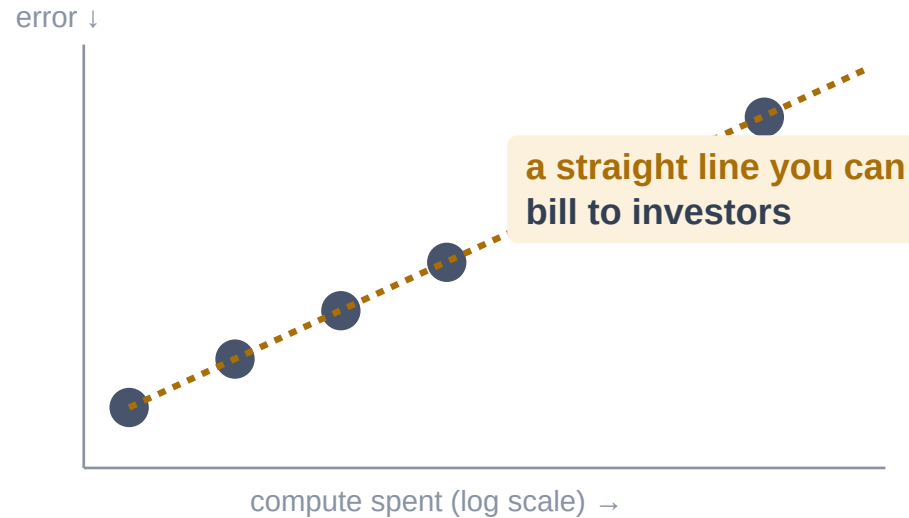
Grow model, data, and compute together: error falls **smoothly and predictably**, across many orders of magnitude.

Scaling laws: progress becomes purchasable



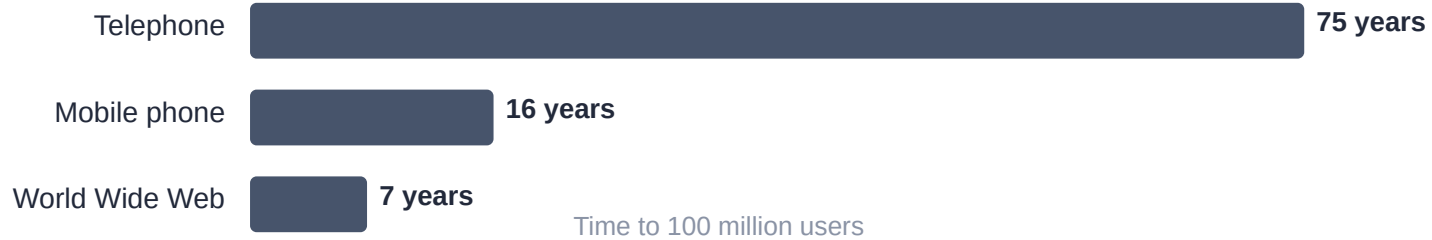
Capability became **forecastable, therefore purchasable**. The arms race was every player's rational response to the same graph.

Scaling laws: progress becomes purchasable



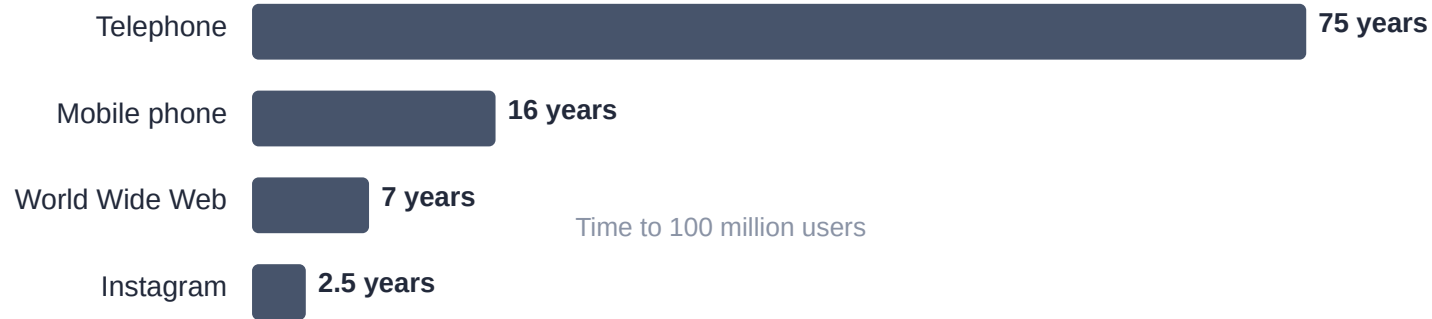
GPT-3 added the surprise: abilities nobody engineered: **emergence**. Bigger did qualitatively more, **ahead of our understanding of why**. (The soup pot: past certain sizes, flavours appear nobody added.)

November 2022

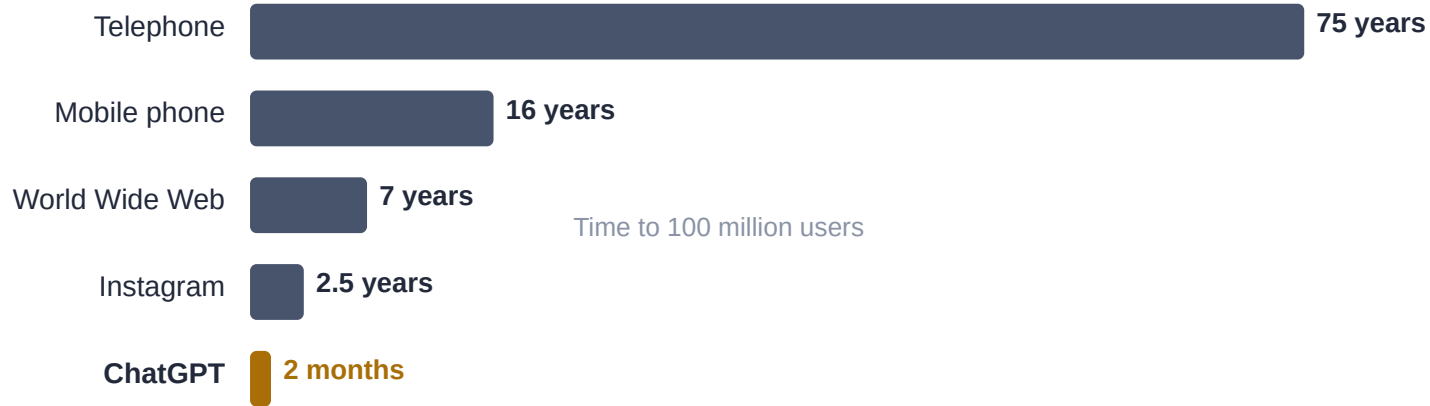


The engine had existed for **two years** as a specialist curiosity.

November 2022



Two additions:
RLHF (tune the model against learned human preferences) and a **chat box**.



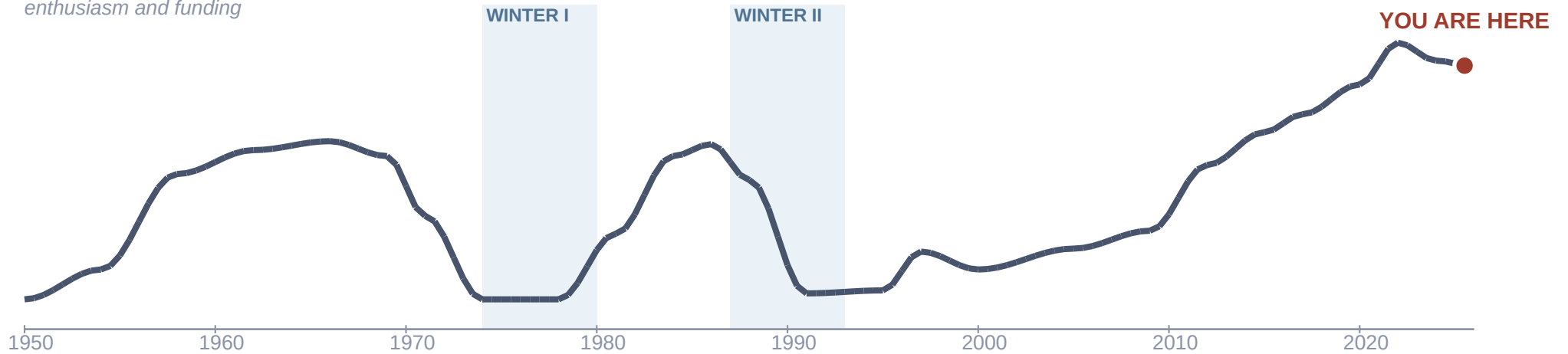
One hundred million users in two months. Capability and adoption are separate achievements; the ELIZA effect is now a **design responsibility**.

PART 11

The walls and the two turns

Where you come in · 2023 to now

enthusiasm and funding



NOW

The scaling recipe meets four walls

the
scaling
recipe



COST

hundreds of millions
per frontier run

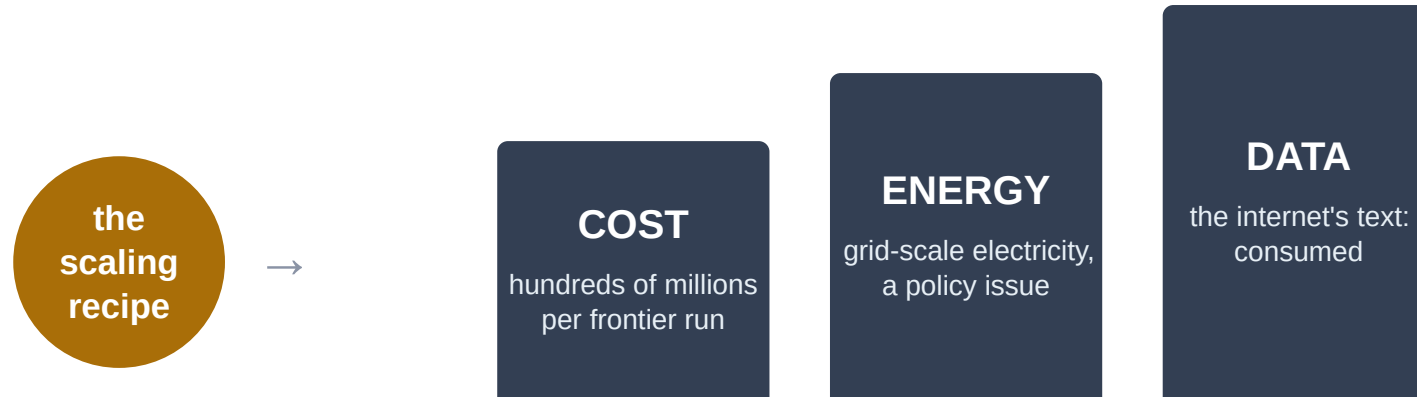
Cost: frontier runs in the hundreds of millions. 2012's frontier fit a student's desktop; today's fits a handful of companies and states.

The scaling recipe meets four walls



Energy: data-centre electricity is now a grid-operator and government topic. Week 9: sustainability as a **budget**.

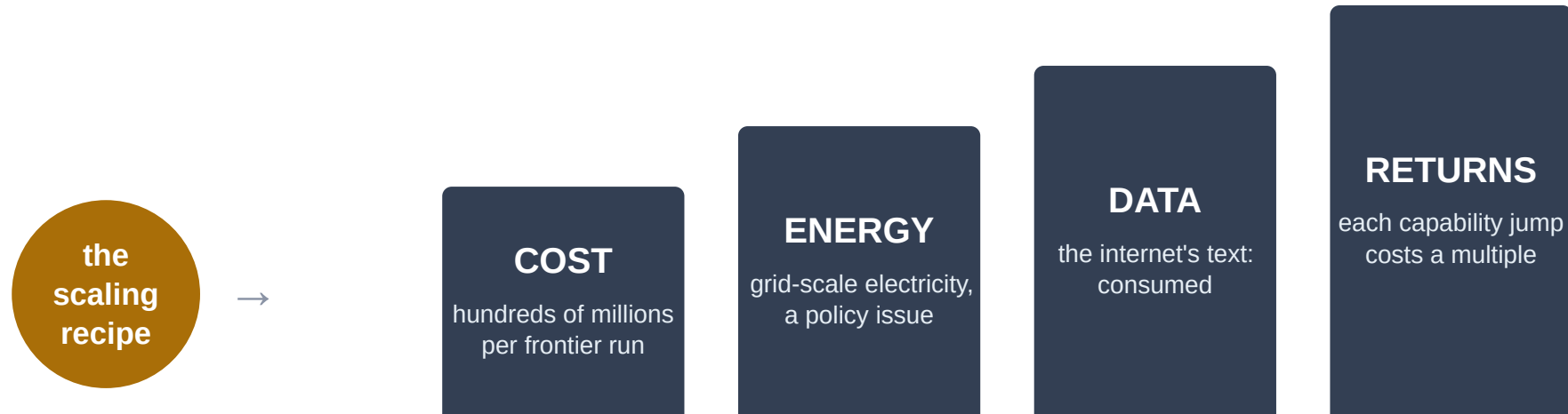
The scaling recipe meets four walls



Data: the internet's accumulated text: consumed in about five years. The first 2012 ingredient to run short.

NOW

The scaling recipe meets four walls



Diminishing returns: the curves flatten. Each equal jump costs a multiple. Not broken: **expensive.**

The field's two answers

Toward science · build known physics **into** models: if the law is built in, the data bill collapses. PINNs, symmetry, AlphaFold at Nobel level · Weeks 5, 11

The field's two answers

Toward science · build known physics **into** models: if the law is built in, the data bill collapses. PINNs, symmetry, AlphaFold at Nobel level · Weeks 5, 11

Toward lean · more capability per parameter, joule, euro. distillation, quantization, LoRA, state-space models, test-time reasoning · Weeks 8, 9, your project

The field's two answers

Toward science · build known physics **into** models: if the law is built in, the data bill collapses. PINNs, symmetry, AlphaFold at Nobel level · Weeks 5, 11

Toward lean · more capability per parameter, joule, euro. distillation, quantization, LoRA, state-space models, test-time reasoning · Weeks 8, 9, your project

Early 2025: **DeepSeek** ships near-frontier quality at a reported fraction of assumed cost. Markets convulse; *spend = capability* cracks in a week. And note: equations entering learning systems = **the two families of 1956, recombining on your watch.**

The module thesis

KEY IDEA

Scaling **worked**. Scaling is **hitting walls**: cost, energy, data, returns. The frontier turns **toward science** and **toward lean**.

The module thesis

KEY IDEA

Scaling **worked**. Scaling is **hitting walls**: cost, energy, data, returns. The frontier turns **toward science** and **toward lean**.

Every technique, every headline, every week:

more brute force?

more science?

more lean?

So... a third winter?

1 • The cycle is real, and this looks like a top.

So... a third winter?

1 • The cycle is real, and this looks like a top. **2** • Winters punish **promises, not science**.
The techniques compounded through both.

So... a third winter?

1 · The cycle is real, and this looks like a top. **2** · Winters punish **promises, not science**. The techniques compounded through both. **3** · This cycle has what 1974 and 1987 lacked: **revenue and daily use**. Busts of that shape ruin investors, not technologies.

So... a third winter?

1 • The cycle is real, and this looks like a top. **2** • Winters punish **promises, not science**. The techniques compounded through both. **3** • This cycle has what 1974 and 1987 lacked: **revenue and daily use**. Busts of that shape ruin investors, not technologies.

KEY IDEA

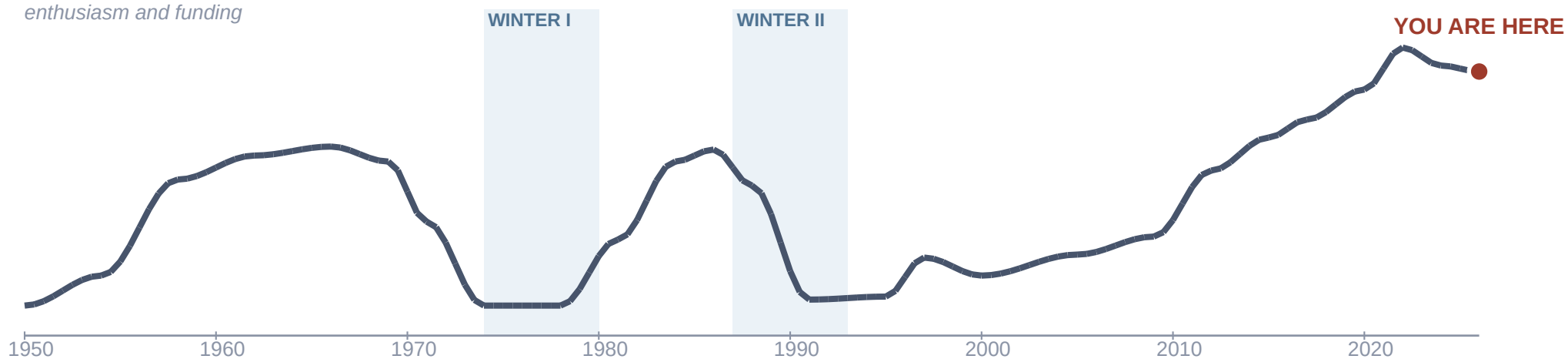
The defensible position is **calibrated uncertainty, argued**. Anyone certain in either direction is selling something. Your Week 12 forecast is Week 1's real exam, sat eleven weeks late.

PART 12

Getting set up

Python, Colab, GitHub, HuggingFace, Ollama · about 45 minutes, once

enthusiasm and funding



DO NOT INSTALL ALL OF THEM TODAY

Five tools, and what each is actually for

Python 3.11+
a virtual environment
Git + GitHub
Google Colab
HuggingFace
Ollama

the language everything else runs on
keeps this module's packages away from everything else on your machine
where every deliverable lives, including the Week 12 project
a free GPU for when your laptop is not enough
where the models and datasets come from
runs models entirely on your own machine, no API key

today
today
today
Weeks 7–9
Week 3
Weeks 7, 9

DO NOT INSTALL ALL OF THEM TODAY

Five tools, and what each is actually for

Python 3.11+
a virtual environment
Git + GitHub
Google Colab
HuggingFace
Ollama

the language everything else runs on
keeps this module's packages away from everything else on your machine
where every deliverable lives, including the Week 12 project
a free GPU for when your laptop is not enough
where the models and datasets come from
runs models entirely on your own machine, no API key

today
today
today
Weeks 7–9
Week 3
Weeks 7, 9

KEY IDEA

Only the first three are needed to leave this room working. The others have deadlines, and the deadlines are weeks away. Installing PyTorch five minutes before a lab is how people miss the lab.

INSTALL ONCE, THEN STOP THINKING ABOUT IT

Python, and the one rule that saves a week

terminal

bash

```
python3 --version      # macOS / Linux      -> need 3.11 or newer
python --version        # Windows
```

INSTALL ONCE, THEN STOP THINKING ABOUT IT

Python, and the one rule that saves a week

terminal

bash

```
python3 --version      # macOS / Linux      -> need 3.11 or newer
python  --version      # Windows
```

Windows

Microsoft Store, or python.org. On the python.org installer, **tick “Add python.exe to PATH”** on the first screen.

macOS

brew install python@3.12, or python.org. Do **not** rely on the /usr/bin/python3 Apple ships.

Linux

sudo apt install python3 python3-venv python3-pip. The venv package is separate and easy to miss.

INSTALL ONCE, THEN STOP THINKING ABOUT IT

Python, and the one rule that saves a week

terminal

bash

```
python3 --version      # macOS / Linux      -> need 3.11 or newer
python --version        # Windows
```

Windows

Microsoft Store, or python.org. On the python.org installer, **tick “Add python.exe to PATH”** on the first screen.

macOS

brew install python@3.12, or python.org. Do **not** rely on the /usr/bin/python3 Apple ships.

Linux

sudo apt install python3 python3-venv python3-pip. The venv package is separate and easy to miss.

KEY IDEA

Never pip install into your system Python. When two modules need different versions of the same library, and they will, the virtual environment is what stops them fighting.

THREE COMMANDS, THEN A PREFIX IN YOUR PROMPT

The virtual environment

terminal · from inside week1-lab/

bash

```
python3 -m venv .venv          # create it, once

source .venv/bin/activate      # activate: macOS / Linux
.venv\Scripts\activate         # activate: Windows PowerShell

pip install -r requirements.txt # core packages only: small and fast
```

THREE COMMANDS, THEN A PREFIX IN YOUR PROMPT

The virtual environment

```
terminal · from inside week1-lab/
```

```
bash
```

```
python3 -m venv .venv          # create it, once

source .venv/bin/activate      # activate: macOS / Linux
.venv\Scripts\activate         # activate: Windows PowerShell

pip install -r requirements.txt # core packages only: small and fast
```

```
prompt
```

```
(.venv) you@laptop week1-lab %
```

That `(.venv)` prefix is the entire point: it tells you **which Python you are about to run**. If it disappears when you open a new terminal, activate again. Nothing is broken.

THREE COMMANDS, THEN A PREFIX IN YOUR PROMPT

The virtual environment

```
terminal · from inside week1-lab/
```

```
bash
```

```
python3 -m venv .venv          # create it, once

source .venv/bin/activate      # activate: macOS / Linux
.venv\Scripts\activate         # activate: Windows PowerShell

pip install -r requirements.txt # core packages only: small and fast
```

```
prompt
```

```
(.venv) you@laptop week1-lab %
```

That `(.venv)` prefix is the entire point: it tells you **which Python you are about to run**. If it disappears when you open a new terminal, activate again. Nothing is broken.

There is a one-command version. `bash setup.sh` creates the environment, installs the core packages, and runs the check. `bash setup.sh --full` adds the PyTorch stack later, when you have the disk space.

SETUP_CHECK.PY · IT NEVER CRASHES

Verify, before you trust

```
$ python setup_check.py
```


Verify, before you trust

```
$ python setup_check.py
```

```
console
```

```
[ OK ] Python 3.12.3 on Linux x86_64
[ OK ] numpy / pandas / sklearn 2.4.4 · 3.0.2 · 1.8.0
[ NOTE ] torch not installed (needed from Week 3)
[ NOTE ] huggingface_hub not installed, bash setup.sh --full
[ NOTE ] Ollama not installed (needed Week 7)
[ NOTE ] git identity not set, git config --global user.name "You"
```

```
6 ready · 11 to sort out later · 0 blocking
```

```
Nothing blocking. You are ready for the timeline exercise.
```

The number that matters is **blocking**. The eleven “sort out later” items have deadlines in Weeks 3 and 7, not today.

Git and GitHub: your repo is the deliverable

terminal

bash

```
git config --global user.name "Your Name"
git config --global user.email "you@setu.ie" # your student address

git init
git add .
git commit -m "Week 1: environment and timeline lab"
git branch -M main
git remote add origin https://github.com/YOURNAME/ai-evolution-2026.git
git push -u origin main
```

Git and GitHub: your repo is the deliverable

terminal

bash

```
git config --global user.name "Your Name"
git config --global user.email "you@setu.ie" # your student address

git init
git add .
git commit -m "Week 1: environment and timeline lab"
git branch -M main
git remote add origin https://github.com/YOURNAME/ai-evolution-2026.git
git push -u origin main
```

GitHub stopped accepting passwords over HTTPS in 2021. When git asks for a password it wants a **personal access token** (Settings → Developer settings), or use `gh auth login`, or set up an SSH key. Whichever you choose, do it once and never think about it again.

Git and GitHub: your repo is the deliverable

terminal

bash

```
git config --global user.name "Your Name"
git config --global user.email "you@setu.ie" # your student address

git init
git add .
git commit -m "Week 1: environment and timeline lab"
git branch -M main
git remote add origin https://github.com/YOURNAME/ai-evolution-2026.git
git push -u origin main
```

GitHub stopped accepting passwords over HTTPS in 2021. When git asks for a password it wants a **personal access token** (Settings → Developer settings), or use `gh auth login`, or set up an SSH key. Whichever you choose, do it once and never think about it again.

KEY IDEA

A .gitignore ships in the lab folder: it keeps .venv/, caches, model weights and .env out of your repository. **Never commit a token.** Week 2 will explain, at length, why what you record matters.

Colab: the escape hatch, not the default

first cell of any Colab notebook

python

```
!git clone https://github.com/YOURNAME/ai-evolution-2026.git
%cd ai-evolution-2026/week1-lab
!pip install -q -r requirements.txt
```

Colab: the escape hatch, not the default

first cell of any Colab notebook

python

```
!git clone https://github.com/YOURNAME/ai-evolution-2026.git  
%cd ai-evolution-2026/week1-lab  
!pip install -q -r requirements.txt
```

To get an accelerator

Runtime → Change runtime type → T4 GPU

It forgets everything

Colab resets on disconnect. Anything you want to keep gets pushed to GitHub.

Colab: the escape hatch, not the default

first cell of any Colab notebook

python

```
!git clone https://github.com/YOURNAME/ai-evolution-2026.git  
%cd ai-evolution-2026/week1-lab  
!pip install -q -r requirements.txt
```

To get an accelerator

Runtime → Change runtime type → T4 GPU

It forgets everything

Colab resets on disconnect. Anything you want to keep gets pushed to GitHub.

Use it when your own machine cannot cope, realistically **Weeks 7 to 9**. For Weeks 1 to 5 a laptop is entirely sufficient, and working locally teaches you more.

HuggingFace: an account, a token, a cache

terminal

bash

```
pip install -U huggingface_hub
hf auth login          # paste a READ token from Settings -> Access Tokens
hf auth whoami         # confirms who you are
```

HuggingFace: an account, a token, a cache

terminal

bash

```
pip install -U huggingface_hub
hf auth login          # paste a READ token from Settings -> Access Tokens
hf auth whoami         # confirms who you are
```

The CLI was called `huggingface-cli` until 2025 and is now `hf`. The old name still works but warns. Most tutorials you find online still use it: `huggingface-cli login` is `hf auth login`.

HuggingFace: an account, a token, a cache

terminal

bash

```
pip install -U huggingface_hub
hf auth login          # paste a READ token from Settings -> Access Tokens
hf auth whoami         # confirms who you are
```

The CLI was called `huggingface-cli` until 2025 and is now `hf`. The old name still works but warns. Most tutorials you find online still use it: `huggingface-cli login` is `hf auth login`.

terminal

bash

```
hf cache scan          # downloads go to ~/.cache/huggingface, and grow
hf cache delete        # reclaim the disk when it fills
```

HuggingFace: an account, a token, a cache

terminal

bash

```
pip install -U huggingface_hub
hf auth login          # paste a READ token from Settings -> Access Tokens
hf auth whoami         # confirms who you are
```

The CLI was called `huggingface-cli` until 2025 and is now `hf`. The old name still works but warns. Most tutorials you find online still use it: `huggingface-cli login` is `hf auth login`.

terminal

bash

```
hf cache scan          # downloads go to ~/.cache/huggingface, and grow
hf cache delete        # reclaim the disk when it fills
```

Gated models (Llama, for instance) need you to accept a licence on the model page while logged in. You meet that in Week 7, not today.

NO API KEY. NO ACCOUNT. NOTHING LEAVES THE LAPTOP.

Ollama: a model on your own machine

terminal

bash

```
# macOS / Windows: installer from https://ollama.com
```

```
curl -fsSL https://ollama.com/install.sh | sh # Linux
```

```
ollama serve # start the background service
```

```
ollama pull llama3.2:1b # about 1.3 GB, start small
```

```
ollama run llama3.2:1b "In two sentences, what was the AI winter of the 1970s?"
```

NO API KEY. NO ACCOUNT. NOTHING LEAVES THE LAPTOP.

Ollama: a model on your own machine

terminal

bash

```
# macOS / Windows: installer from https://ollama.com
curl -fsSL https://ollama.com/install.sh | sh          # Linux

ollama serve                # start the background service
ollama pull llama3.2:1b      # about 1.3 GB, start small
ollama run llama3.2:1b "In two sentences, what was the AI winter of the 1970s?"
```

the same thing, over HTTP, this is what Week 10 will talk to

bash

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2:1b", "prompt": "Why did AlexNet matter?", "stream": false
}'
```

NO API KEY. NO ACCOUNT. NOTHING LEAVES THE LAPTOP.

Ollama: a model on your own machine

terminal

bash

```
# macOS / Windows: installer from https://ollama.com
curl -fsSL https://ollama.com/install.sh | sh # Linux

ollama serve # start the background service
ollama pull llama3.2:1b # about 1.3 GB, start small
ollama run llama3.2:1b "In two sentences, what was the AI winter of the 1970s?"
```

the same thing, over HTTP, this is what Week 10 will talk to

bash

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2:1b", "prompt": "Why did AlexNet matter?", "stream": false
}'
```

Sizing. A 1B model wants about 2 GB of RAM; 7–8B wants about 8 GB; beyond that you want a GPU. Start at 1b and only grow if it is comfortable.

NO API KEY. NO ACCOUNT. NOTHING LEAVES THE LAPTOP.

Ollama: a model on your own machine

terminal

bash

```
# macOS / Windows: installer from https://ollama.com
curl -fsSL https://ollama.com/install.sh | sh # Linux

ollama serve # start the background service
ollama pull llama3.2:1b # about 1.3 GB, start small
ollama run llama3.2:1b "In two sentences, what was the AI winter of the 1970s?"
```

the same thing, over HTTP, this is what Week 10 will talk to

bash

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2:1b", "prompt": "Why did AlexNet matter?", "stream": false
}'
```

Sizing. A 1B model wants about 2 GB of RAM; 7–8B wants about 8 GB; beyond that you want a GPU. Start at 1b and only grow if it is comfortable.

KEY IDEA

When a 1.3 GB file answers you, on your own laptop, with the wifi off: **Week 9's lean turn stops being a slide.** That is the single most persuasive demo in this module, and it costs you five minutes today.

When it goes wrong

Symptom	Cause	Fix
python: command not found	not on PATH, or it is python3	try python3; on Windows reinstall ticking “Add to PATH”
externally-managed-environment	installing into system Python	activate the venv. This error is the rule being enforced
(.venv) vanished	new terminal window	activate again; nothing is broken
ModuleNotFoundError after installing	the editor is using a different interpreter	VS Code → Python: Select Interpreter → .venv/bin/python
password authentication was removed	git wants a token	personal access token, gh auth login, or SSH
401 Unauthorized from the Hub	not logged in, or gated model	hf auth login, and accept the licence on the model page
Ollama connection refused on 11434	service not running	ollama serve in a second terminal
Colab “cannot connect to GPU”	free tier busy or quota used	wait, or carry on with CPU: Weeks 1–5 do not need one

Before next week

- ☐ `python setup_check.py` reports **0 blocking**
- ☐ A **private GitHub repository** exists, and this folder is pushed to it
- ☐ `hf auth whoami` returns your username
- ☐ `ollama run llama3.2:1b "hello"` answers **on your own machine**
- ☐ You have opened one **Colab** notebook and watched a cell run

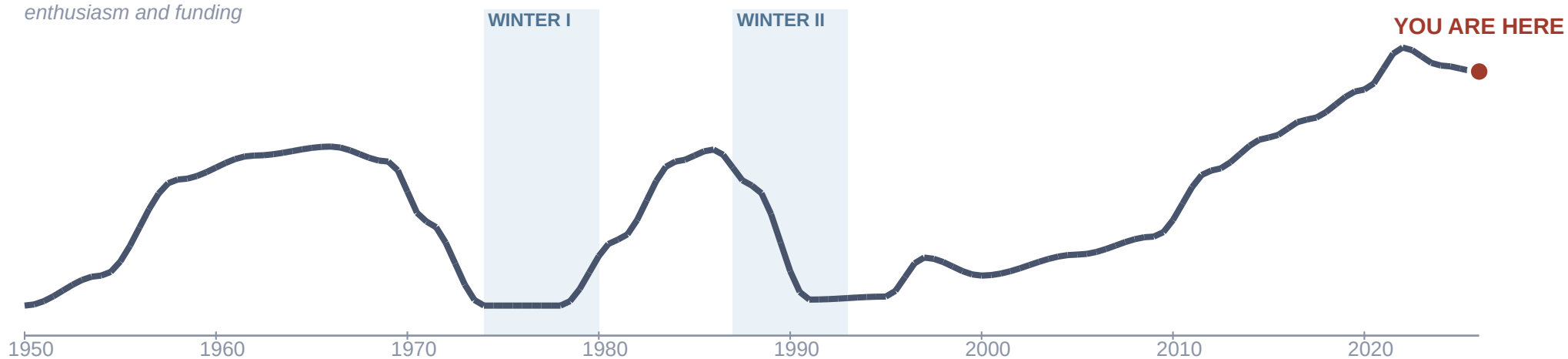
The full walkthrough, per operating system, with the troubleshooting table, is `INSTALL.md` in the lab folder. Bring the output of `setup_check.py` to the lab if anything is stuck.

PART 13

The lab, in code

Environment, the timeline as data, and the thesis as an experiment

enthusiasm and funding



Check the machine before you blame the model

week1-lab/setup_check.py

python

```
def check_accelerator():
    import torch
    if torch.cuda.is_available():
        record(OK, "accelerator", torch.cuda.get_device_name(0))
    elif torch.backends.mps.is_available():
        record(OK, "accelerator", "Apple MPS")
    else:
        record(WARN, "accelerator", "CPU only: fine to Week 5, use Colab later")
```

```
$ python setup_check.py
```

Check the machine before you blame the model

week1-lab/setup_check.py

python

```
def check_accelerator():
    import torch
    if torch.cuda.is_available():
        record(OK, "accelerator", torch.cuda.get_device_name(0))
    elif torch.backends.mps.is_available():
        record(OK, "accelerator", "Apple MPS")
    else:
        record(WARN, "accelerator", "CPU only: fine to Week 5, use Colab later")
```

```
$ python setup_check.py
```

console

```
[ OK ] Python 3.12.3 on Linux x86_64
[ OK ] numpy / pandas / sklearn 2.4.4 · 3.0.2 · 1.8.0
[ NOTE ] torch not installed (needed from Week 3)
[ NOTE ] Ollama not installed (needed Week 7)
[ NOTE ] git identity not set
```

8 ready · 9 to sort out later · 0 blocking

The timeline is a dataset, not a picture

week1-lab/timeline_data.py

python

```
# each milestone carries the two things the
# lecture actually argues about
(1958, "Perceptron", "system",
     "connectionist", "architecture",
     "First machine that learns from error."),
(2012, "AlexNet", "system",
     "connectionist", "architecture",
     "26% to 15%. All three, at once."),

df = load()          # -> tidy DataFrame
df.family.value_counts()
```


The timeline is a dataset, not a picture

week1-lab/timeline_data.py

python

```
# each milestone carries the two things the
# lecture actually argues about
(1958, "Perceptron", "system",
     "connectionist", "architecture",
     "First machine that learns from error."),
(2012, "AlexNet", "system",
     "connectionist", "architecture",
     "26% to 15%. All three, at once."),
```

```
df = load()          # -> tidy DataFrame
df.family.value_counts()
```

out

62 milestones, 1843 to 2025

family	
connectionist	27
symbolic	17
neither	10
statistical	8

Tagging the record makes the pendulum of Part 3 **queryable** instead of asserted. Exercise 1 asks you to plot it and find the crossover yourself.

When did each ingredient actually arrive?

week1-lab/timeline_data.py

python

```
# Defensible, not definitive. Argue with these: that is Exercise 2.
SUPPLY = {
    "architecture": (1986, "Backprop: depth becomes trainable in principle."),
    "compute":      (2007, "CUDA: commodity gaming silicon becomes general-purpose."),
    "data":         (2009, "ImageNet: labelled data at unprecedented scale."),
}
ingredients_available()          # boolean per year, plus all_three
```

When did each ingredient actually arrive?

week1-lab/timeline_data.py

python

```
# Defensible, not definitive. Argue with these: that is Exercise 2.
SUPPLY = {
    "architecture": (1986, "Backprop: depth becomes trainable in principle."),
    "compute":      (2007, "CUDA: commodity gaming silicon becomes general-purpose."),
    "data":         (2009, "ImageNet: labelled data at unprecedented scale."),
}
ingredients_available()          # boolean per year, plus all_three
```

out

```
All three available from:      2009
First system to combine them: AlexNet, 2012
```

The gap is 3 years. Availability is not assembly.

When did each ingredient actually arrive?

```
week1-lab/timeline_data.py
```

```
python
```

```
# Defensible, not definitive. Argue with these: that is Exercise 2.  
SUPPLY = {  
    "architecture": (1986, "Backprop: depth becomes trainable in principle."),  
    "compute":      (2007, "CUDA: commodity gaming silicon becomes general-purpose."),  
    "data":         (2009, "ImageNet: labelled data at unprecedented scale."),  
}  
ingredients_available()          # boolean per year, plus all_three
```

```
out
```

```
All three available from:      2009  
First system to combine them: AlexNet, 2012
```

```
The gap is 3 years. Availability is not assembly.
```

KEY IDEA

A convolutional architecture existed in 1979; a way to train one in 1986; nobody could **feed** it until 2009 or **afford** it until 2007. The three-year gap to AlexNet is the most interesting number on this slide.

The thesis, as an experiment

week1-lab/three_ingredients.py

python

```
SETTINGS = {                                # LOW / HIGH for each ingredient
    "data": {"low": 120, "high": 1400},      # training examples
    "architecture": {"low": (2,), "high": (128, 64)}, # hidden units
    "compute": {"low": 8, "high": 600},      # optimiser iterations
}

def run_one(data_hi, arch_hi, compute_hi, seed=0):
    X, y = load_digits(return_X_y=True)      # ships with sklearn
    ...
    clf = MLPClassifier(hidden_layer_sizes=arch, max_iter=iters)
    return clf.score(sc.transform(Xte), yte)
```

The thesis, as an experiment

week1-lab/three_ingredients.py

python

```
SETTINGS = {                                # LOW / HIGH for each ingredient
    "data": {"low": 120, "high": 1400},      # training examples
    "architecture": {"low": (2,), "high": (128, 64)}, # hidden units
    "compute": {"low": 8, "high": 600},      # optimiser iterations
}

def run_one(data_hi, arch_hi, compute_hi, seed=0):
    X, y = load_digits(return_X_y=True)      # ships with sklearn
    ...
    clf = MLPClassifier(hidden_layer_sizes=arch, max_iter=iters)
    return clf.score(sc.transform(Xte), yte)
```

```
$ python three_ingredients.py
```

Eight combinations. Three seeds each. About sixty seconds. The lecture asserted that 2012 was a coincidence of supply; this **measures** it.

Every partial combination stalls. Measured.

out · held-out accuracy on sklearn digits

data	arch	comp	n_train	hidden	iters	train	TEST	era
low	low	low	120	(2,)	8	0.189	0.190	nothing at all
HIGH	low	low	1257	(2,)	8	0.405	0.402	data alone: the 2000s web
low	low	HIGH	120	(2,)	600	0.719	0.451	compute alone: Deep Blue 1997
HIGH	low	HIGH	1257	(2,)	600	0.708	0.672	data+compute: ImageNet 2010
low	HIGH	low	120	(128, 64)	8	1.000	0.862	architecture alone: perceptron
low	HIGH	HIGH	120	(128, 64)	600	1.000	0.881	arch+compute: fast overfitting
HIGH	HIGH	low	1257	(128, 64)	8	0.999	0.962	data+arch: 1990s nets, too slow
HIGH	HIGH	HIGH	1257	(128, 64)	600	1.000	0.970	ALL THREE: AlexNet, 2012

Read the **era** column against the accuracy. Each partial combination has a name from this lecture, and each one stalls where history says it stalled.

Every partial combination stalls. Measured.

```
out · held-out accuracy on sklearn digits
```

data	arch	comp	n_train	hidden	iters	train	TEST	era
low	low	low	120	(2,)	8	0.189	0.190	nothing at all
HIGH	low	low	1257	(2,)	8	0.405	0.402	data alone: the 2000s web
low	low	HIGH	120	(2,)	600	0.719	0.451	compute alone: Deep Blue 1997
HIGH	low	HIGH	1257	(2,)	600	0.708	0.672	data+compute: ImageNet 2010
low	HIGH	low	120	(128, 64)	8	1.000	0.862	architecture alone: perceptron
low	HIGH	HIGH	120	(128, 64)	600	1.000	0.881	arch+compute: fast overfitting
HIGH	HIGH	low	1257	(128, 64)	8	0.999	0.962	data+arch: 1990s nets, too slow
HIGH	HIGH	HIGH	1257	(128, 64)	600	1.000	0.970	ALL THREE: AlexNet, 2012

Look at row five. Train 1.000, test 0.862: a big model with a small dataset does not fail to learn, it **memorises**. That is the 1990s in one line, and Week 3's overfitting lecture in advance.

Every partial combination stalls. Measured.

out · held-out accuracy on sklearn digits

data	arch	comp	n_train	hidden	iters	train	TEST	era
low	low	low	120	(2,)	8	0.189	0.190	nothing at all
HIGH	low	low	1257	(2,)	8	0.405	0.402	data alone: the 2000s web
low	low	HIGH	120	(2,)	600	0.719	0.451	compute alone: Deep Blue 1997
HIGH	low	HIGH	1257	(2,)	600	0.708	0.672	data+compute: ImageNet 2010
low	HIGH	low	120	(128, 64)	8	1.000	0.862	architecture alone: perceptron
low	HIGH	HIGH	120	(128, 64)	600	1.000	0.881	arch+compute: fast overfitting
HIGH	HIGH	low	1257	(128, 64)	8	0.999	0.962	data+arch: 1990s nets, too slow
HIGH	HIGH	HIGH	1257	(128, 64)	600	1.000	0.970	ALL THREE: AlexNet, 2012

KEY IDEA

No single ingredient carries the result; the jump is in the **combination**. Exercise 4: find a setting where **more compute makes test accuracy worse**, and explain it in one sentence.

First contact with the runtimes

week1-lab/hello_models.py

python

```
def ollama_hello(model=None, prompt=None):  
    """One local generation: no API key, no data leaving the machine."""  
    if not ollama_running():  
        print("install: https://ollama.com   then: ollama pull llama3.2:1b")  
        return None  
    payload = json.dumps({"model": model, "prompt": prompt, "stream": False})  
    with urllib.request.urlopen(req, timeout=120) as r:  
        return json.loads(r.read())["response"]
```

First contact with the runtimes

week1-lab/hello_models.py

python

```
def ollama_hello(model=None, prompt=None):  
    """One local generation: no API key, no data leaving the machine."""  
    if not ollama_running():  
        print("install: https://ollama.com   then: ollama pull llama3.2:1b")  
        return None  
    payload = json.dumps({"model": model, "prompt": prompt, "stream": False})  
    with urllib.request.urlopen(req, timeout=120) as r:  
        return json.loads(r.read())["response"]
```

console · nothing installed yet, and it says so politely

```
Ollama is not responding on localhost:11434.  
install:  https://ollama.com  
start:    ollama serve  
a model:  ollama pull llama3.2:1b      (about 1.3 GB)
```

Get this working before Week 7. When a 1.3 GB file answers you on your own laptop, **Week 9's lean turn stops being a slide.**

Seven questions for any AI headline

- 1 What did it actually do, and in what world: pool or ocean?
- 2 Would it survive a benchmark a rival can run?
- 3 Whose supply made it possible, and is it durable?
- 4 Engine, or interface?

Seven questions for any AI headline

- 1 What did it actually do, and in what world: pool or ocean?
- 2 Would it survive a benchmark a rival can run?
- 3 Whose supply made it possible, and is it durable?
- 4 Engine, or interface? 5 Who is speaking, and what are they selling?
- 6 Which version of the idea does the criticism reach?
- 7 Brute force, science, or lean?

Seven questions for any AI headline

- 1** What did it actually do, and in what world: pool or ocean?
- 2** Would it survive a benchmark a rival can run?
- 3** Whose supply made it possible, and is it durable?
- 4** Engine, or interface? **5** Who is speaking, and what are they selling?
- 6** Which version of the idea does the criticism reach?
- 7** Brute force, science, or lean?

Standing exercise: one headline, seven questions, under 200 words, every week. Week 6: quiz prep. Week 12: a paragraph of your critical brief.

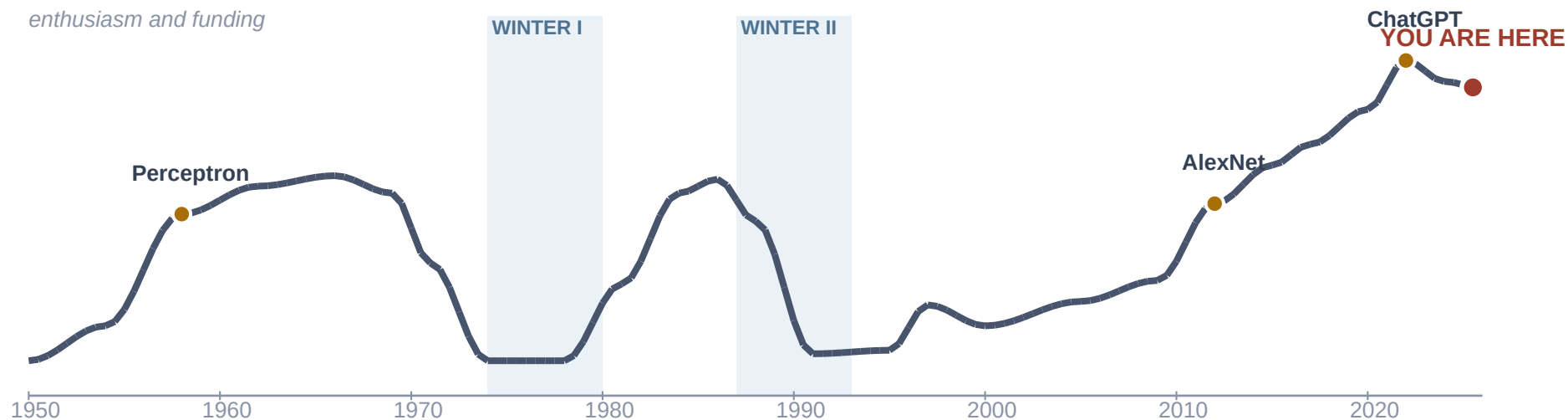
Before the lab

- Interactive companion: scrub the wave; predict all 8 ingredient toggles
- Booklet spine: Chapters 1, 5, 6, 9, 10
- Break an ELIZA clone; post the killer question

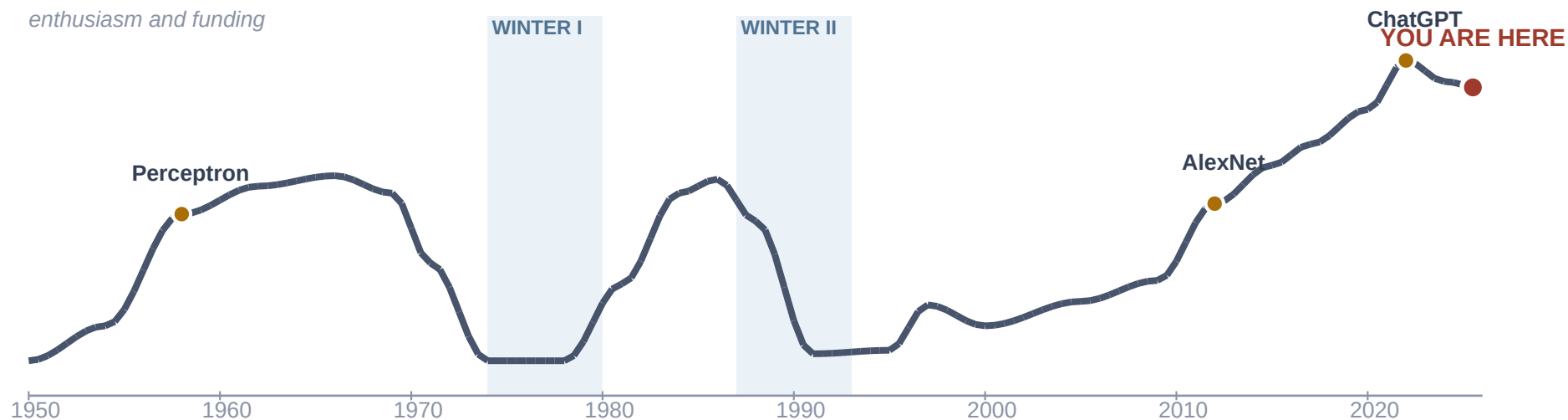
Assessment radar

- Week 6 quiz (50%): this lecture + booklet
- Reliable: the cycle · XOR · three diseases · three ingredients · Deep Blue vs AlphaGo · the thesis
- Booklet Ch. 17: 43 practice questions

Lab: environment setup. The darts loop, industrialised, awaits in Week 3.



The beach was always real.



The beach was always real. Watch the promises.

Next week: Responsible AI, before we build anything.